Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

PiHub

A Matter-Enabled IoT Smart Home Hub

Project Engineering

Year 4

Tremain Mannion

Bachelor of Engineering (Honours) in Software and Electronic Engineering

Atlantic Technological University

2025 / 2026

Declaration

This project is presented in partial fulfilment of the requirements for the degree of Bachelor of Engineering (Honours) in Software and Electronic Engineering at Atlantic Technological University.

This project is my own work, except where otherwise accredited. Where the work of others has been used or incorporated during this project, this is acknowledged and referenced in accordance with IEEE citation style.

AI tools (Claude) were used as an aid during the writing of this report. All technical content, design decisions, implementation work, and conclusions are entirely my own. AI assistance was used to help structure and articulate ideas, not to replace original thought.

Signed: Tremain Mannion_____

Date: 27-04-2026_____

Acknowledgements

I would like to thank all the lecturers and staff at Atlantic Technological University who supported me throughout this degree. The knowledge and skills I have built up over the past four years have been directly applied throughout this project, and that wouldn't have been possible without the quality of teaching I received.

A particular thank you to my project supervisor Fergus Downey, whose guidance and feedback throughout the year kept me on the right track. His willingness to engage with the technical challenges of the project and point me in the right direction when I needed it made a real difference.

A thanks to Daniel Kelly also. With his 3D printing knowledge and access to 3D printing equipment, it made sourcing and creating all the necessary housings quick and easy.

I would also like to acknowledge my classmates, who were a great source of collaboration and support throughout the project. Sharing ideas, talking through problems, and learning from each other made the whole process a lot more enjoyable.

Table of Contents

Declaration ... 2
Acknowledgements ... 3
Table of Contents ... 4
1. Summary ... 7
2. Poster ... 8
3. Introduction ... 9
3.1 Project Overview ... 9
3.2 Motivation ... 9
3.3 Project Scope ... 9
3.4 Report Structure ... 10
4. Background and Literature Review ... 11
4.1 The IoT Ecosystem and Interoperability Problem ... 11
4.2 The Matter Protocol ... 11
4.2.1 Overview and Governance ... 11
4.2.2 Network Layer and Transport ... 12
4.2.3 Commissioning ... 12
4.2.4 Device Model and Clusters ... 13
4.2.5 Security Model ... 14
4.3 Microcontroller Platform: ESP32-C3 ... 14
4.3.1 Hardware Specification ... 14
4.3.2 ESP-IDF and Esp-matter Framework ... 15
4.3.3 Peripheral Subsystems ... 16
4.4 Sensors and Actuators ... 17
4.4.1 DHT22 Temperature and Humidity Sensor ... 17
4.4.2 WS2812 Addressable RGB LED ... 17
4.5 Backend and Database Technologies ... 17
4.5.1 FastAPI ... 17
4.5.2 PostgreSQL ... 18
4.5.3 SQLAlchemy ... 18
4.6 Frontend Technologies ... 19
4.6.1 React and Vite ... 19
4.6.2 Touch and Kiosk UI Considerations ... 19
4.7 Containerisation and DevOps ... 20
4.7.1 Docker and Docker Compose ... 20
4.7.2 CI/CD with GitHub Actions and GHCR ... 20
4.8 TinyML and On-Device Inference ... 21
4.8.1 TensorFlow and TensorFlow Lite Micro ... 21
4.8.2 Context Classification Approaches ... 21

5. System Architecture 23
5.1 High-Level Architecture Overview 23
5.2 Hardware Architecture 23
5.2.1 Central Hub: Raspberry Pi 4 23
5.2.2 Sensor Node: ESP32-C3 + DHT22 24
5.2.3 Actuator Node: ESP32-C3 + WS2812 LED 24
5.2.4 Network Topology 25
5.3 Software Architecture 25
5.3.1 Backend Service (FastAPI) 25
5.3.2 Matter WebSocket Client (python-matter-server) 26
5.3.3 Database Layer 26
5.3.4 Frontend Application 27
5.3.5 Docker Compose Architecture 28
5.4 Data Flow 28
5.5 Security Architecture 29
6. Project Plan 31
6.1 Project Management Methodology 31
6.2 Work Breakdown and Timeline 31
6.3 Milestones 32
7. Implementation 33
7.1 ESP32-C3 Firmware Development 33
7.1.1 Development Environment 33
7.1.2 DHT22 Sensor Firmware 33
7.1.3 RGB LED Firmware 34
7.2 Backend Implementation 35
7.2.1 Implementation Challenges & Key Decisions 35
7.3 Frontend Implementation 36
7.3.1 Colour Pipeline Implementation 36
7.3.2 Touch and Kiosk Optimisation 37
7.4 Containerisation and Deployment 38
7.5 TinyML On-Device Context Classification 38
7.5.1 Problem Definition and Class Design 38
7.5.2 Data Collection and Synthetic Augmentation 39
7.5.3 Model Architecture and Training 40
7.5.4 Model Quantisation and Firmware Export 41
7.5.5 TFLM Firmware Integration 42
7.5.6 Custom Matter Vendor Cluster - Context Attribute 43
8. Testing and Evaluation 45
8.1 Testing Strategy 45

8.2 Backend Testing - pytest 45
8.3 Frontend Testing - Vitest 47
8.4 System Integration Testing 48
8.5 UI and Kiosk Testing 48
9. Hardware Enclosures 50
9.1 Requirements and Selection Criteria 50
9.2 3D Printing 50
9.3 Final Assembly 51
10. Ethics 52
10.1 Data Privacy 52
10.2 Security and Attack Surface 52
10.3 Environmental Impact 52
10.4 Accessibility 53
11. Conclusion 54
11.1 Summary of Outcomes 54
11.2 Evaluation Against Objectives 54
11.3 Future Work 55
11.4 Reflection 56
Appendix 57
Appendix A - Full Firmware Source Listings 57
Appendix B - PiHub Source Code 57
Appendix C - Docker Compose Files 57
Appendix D - GitHub Actions Workflow 57
Appendix E - TinyML Model Training Script 57
Appendix F - 3D Enclosure Details 57
Appendix G - Jira Project Screenshot Example 58
Appendix H - Weekly Project Logs Example 59
References 60

1. Summary

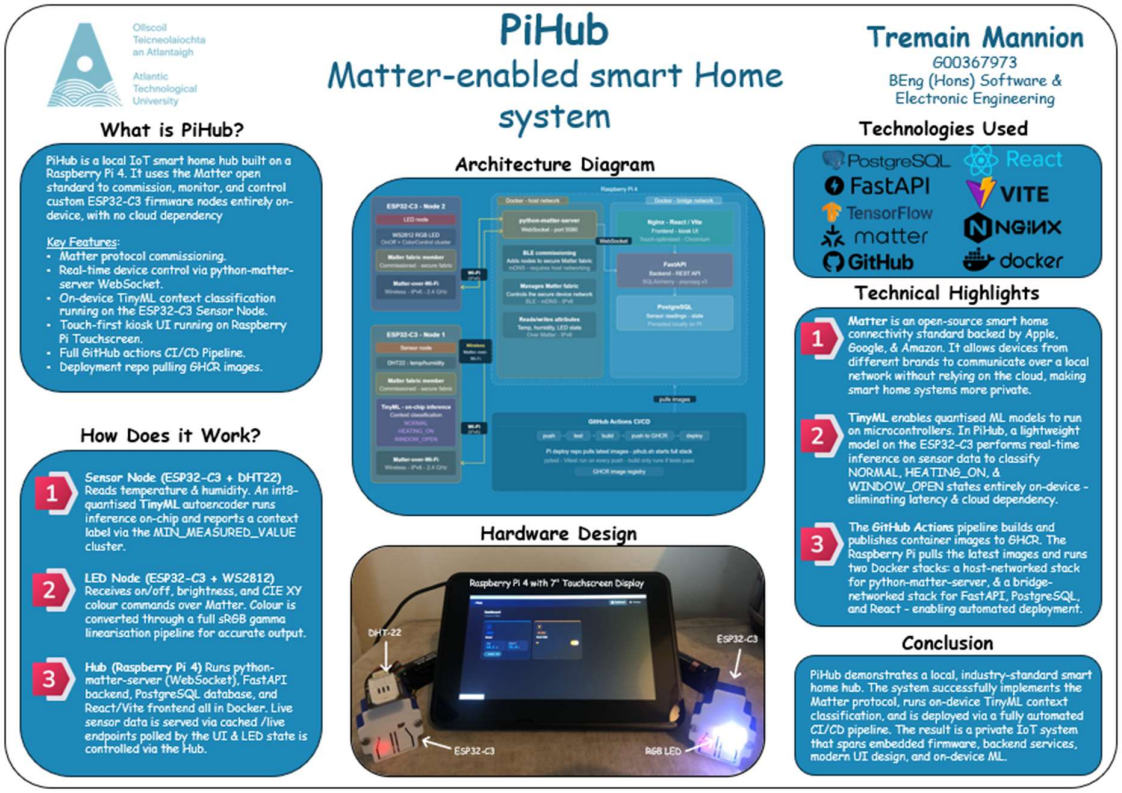
The smart home industry is heavily fragmented, with most devices locked into proprietary ecosystems that require cloud connectivity and brand-specific hardware. PiHub was built to address this directly. The goal of the project was to design and implement a fully open-source, self-hosted smart home hub that uses the Matter protocol to create a secure, interoperable IoT ecosystem that is not tied to any particular brand or platform.

The system was built around a Raspberry Pi 4 acting as the central hub, running a containerised software stack deployed via Docker Compose. Two custom ESP32-C3 nodes were developed using the Esp-matter SDK: a DHT22 temperature and humidity sensor node, and a WS2812 RGB LED actuator node. Both nodes communicate with the hub over Wi-Fi using the Matter protocol, commissioned and managed through python-matter-server. A FastAPI backend handles sensor data and device control, persisting sensor readings to a PostgreSQL database. A React and Vite frontend, served via Nginx, provides a touch-optimised dashboard designed for use on a Raspberry Pi touchscreen in kiosk mode.

I additionally integrated a TinyML context classifier directly onto the ESP32-C3. A three-class feed-forward neural network, trained on synthetic sensor data based on my own home environments and quantised to int8 using TensorFlow Lite Micro, runs inference on the device and classifies the current environmental context as HEATING_ON, NORMAL, or WINDOW_OPEN. The result is exposed to the hub via an unused humidity MIN_MEASURED_VALUE cluster attribute.

The full stack is entirely operational. A GitHub Actions CI/CD pipeline runs automated pytest and Vitest test suites in parallel, setting up a multi-architecture Docker build and push to GHCR on every commit. The project demonstrates that it is entirely feasible to build a production-grade, brand-independent smart home hub using open standards - delivering local control, strong security through Matter's device attestation model, and an extensible foundation for future IoT devices.

2. Poster


Ollscoil Teicneolaíochta an Atlantaigh
Atlantic Technological University
PiHub
Matter-enabled smart Home system
Tremain Mannion
G00367973
BEng (Hons) Software & Electronic Engineering
What is PiHub?
PiHub is a local IoT smart home hub built on a Raspberry Pi 4. It uses the Matter open standard to commission, monitor, and control custom ESP32-C3 firmware nodes entirely on-device, with no cloud dependency
Key Features:
• Matter protocol commissioning.
• Real-time device control via python-matter-server WebSocket.
• On-device TinyML context classification running on the ESP32-C3 Sensor Node.
• Touch-first kiosk UI running on Raspberry Pi Touchscreen.
• Full GitHub actions CI/CD Pipeline.
• Deployment repo pulling GHCR images.
How Does it Work?
1 Sensor Node (ESP32-C3 + DHT22) Reads temperature & humidity. An int8-quantised TinyML autoencoder runs inference on-chip and reports a context label via the MIN_MEASURED_VALUE cluster.
2 LED Node (ESP32-C3 + WS2812) Receives on/off, brightness, and CIE XY colour commands over Matter. Colour is converted through a full sRGB gamma linearisation pipeline for accurate output.
3 Hub (Raspberry Pi 4) Runs python-matter-server (WebSocket), FastAPI backend, PostgreSQL database, and React/Vite frontend all in Docker. Live sensor data is served via cached /live endpoints polled by the UI & LED state is controlled via the Hub.
Architecture Diagram
Hardware Design
Raspberry Pi 4 with 7" Touchscreen Display
DHT-22
ESP32-C3
ESP32-C3
RGB LED
Technologies Used
PostgreSQL
React
FastAPI
VITE
TensorFlow
NGINX
matter
GitHub
docker
Technical Highlights
1 Matter is an open-source smart home connectivity standard backed by Apple, Google, & Amazon. It allows devices from different brands to communicate over a local network without relying on the cloud, making smart home systems more private.
2 TinyML enables quantised ML models to run on microcontrollers. In PiHub, a lightweight model on the ESP32-C3 performs real-time inference on sensor data to classify NORMAL, HEATING_ON, & WINDOW_OPEN states entirely on-device - eliminating latency & cloud dependency.
3 The GitHub Actions pipeline builds and publishes container images to GHCR. The Raspberry Pi pulls the latest images and runs two Docker stacks: a host-networked stack for python-matter-server, & a bridge-networked stack for FastAPI, PostgreSQL, and React - enabling automated deployment.
Conclusion
PiHub demonstrates a local, industry-standard smart home hub. The system successfully implements the Matter protocol, runs on-device TinyML context classification, and is deployed via a fully automated CI/CD pipeline. The result is a private IoT system that spans embedded firmware, backend services, modern UI design, and on-device ML.

3. Introduction

3.1 Project Overview

PiHub is a self-hosted touch-optimised smart home hub built on a Raspberry Pi 4, designed for technically minded users who want full control over their IoT devices without relying on cloud platforms or proprietary ecosystems. It uses the Matter protocol to commission, monitor, and control custom-built ESP32-C3 sensor and actuator nodes over a local Wi-Fi network, with no external service dependencies at runtime.

The core problem PiHub addresses is the fragmentation and cloud-dependency of modern smart home systems. Most commercial solutions require devices to communicate via manufacturer-hosted servers, meaning functionality is tied to third-party uptime, data privacy policies, and continued product support. PiHub replaces this with a fully local, open-standards-based alternative that the user owns and operates entirely.

3.2 Motivation

The smart home market has grown rapidly over the past decade, but with that growth has come significant fragmentation. Devices from different manufacturers rarely work together without a shared cloud platform acting as the middleman, and even then, interoperability is limited. Systems like Apple Home, Google Home, and Amazon Alexa each operate within their own walled gardens, requiring users to commit to a single ecosystem and accept the data privacy and availability trade-offs that come with it.

The emergence of Matter as an open, IP-based IoT standard presented an opportunity to change this. Developed by the Connectivity Standards Alliance and backed by major industry players, Matter was designed specifically to solve the interoperability problem by providing a common protocol that any compliant device can speak, regardless of brand. Building a hub around Matter meant the project could explore what a truly open, vendor-neutral smart home system looks like in practice.

On a personal level, I wanted to use this project as an opportunity to gain hands-on experience with tools and workflows that I knew would be valuable beyond the degree. Working with Docker to containerise and deploy a real multi-service application and using a Raspberry Pi as a genuine deployment target rather than just a development board, were both things I was keen to get practical experience with. The same goes for building a CI/CD pipeline with GitHub Actions - automating the build, test, and deployment process is something I had not done at scale before, and this project gave me a concrete reason to learn it properly.

3.3 Project Scope

The scope of PiHub covers the full design and implementation of a locally hosted, Matter-enabled smart home hub, from embedded firmware through to a touch-optimised web dashboard. The following components fall within the scope of the project:

- Two custom ESP32-C3-DevKitM-1 nodes developed using the ESP-IDF VSCode extension & the Esp-matter SDK - a DHT22 temperature and humidity sensor node, and a WS2812 RGB LED actuator node
- A FastAPI backend providing REST endpoints for device control and sensor data ingestion, integrated with python-matter-server via WebSocket
- A PostgreSQL database for persistent storage of sensor readings

- A React and Vite frontend served via Nginx, optimised for touch input on a Raspberry Pi touchscreen in kiosk mode
- A Docker Compose deployment strategy, split across two stacks to accommodate the network constraints of python-matter-server
- A GitHub Actions CI/CD pipeline automating build, test, and image publishing to GHCR
- A TinyML on-device context classification workstream, running a quantised feed-forward neural network on the ESP32-C3 using TensorFlow Lite Micro
- An automated testing workstream covering backend endpoints with pytest and frontend helper functions with Vitest

The following are explicitly out of scope:

- Matter over Thread - the ESP32-C3 supports Matter over Wi-Fi only, and Thread would have required additional Border Router hardware that was not available for this project
- Multi-user access control - a single-user local dashboard was sufficient for the scope of the project, and the time was better spent on the TinyML workstream
- Voice assistant integration - this was identified as a potential extension but was not implemented within the project timeline

3.4 Report Structure

The remainder of this report is structured as follows:

Section 4 provides background and literature review covering the key technologies and concepts underpinning the project, including the Matter protocol, the ESP32-C3 microcontroller platform, the sensors and actuators used, the backend and frontend technologies, containerisation and DevOps tooling, and TinyML on-device inference.

Section 5 describes the overall system architecture, covering both the hardware topology and the software architecture, including the Docker Compose stack design, data flow through the system, and the security model.

Section 6 outlines the project plan, including the management methodology, timeline, milestones, and risk register.

Section 7 covers the implementation & implementation challenges, walking through the firmware development for both ESP32-C3 nodes, the backend and frontend implementation challenges, the containerisation and CI/CD pipeline, and the TinyML context classification workstream.

Section 8 presents the testing and evaluation strategy and results, covering backend pytest and frontend Vitest test suites, system integration testing, and performance evaluation.

Section 9 covers the hardware enclosures sourced for the ESP32-C3 nodes and Raspberry Pi 4 with the touch screen 2, including the selection criteria and 3D printing process.

Section 10 discusses the ethical considerations relevant to the project.

Section 11 presents the conclusions, evaluating the project outcomes against the original objectives and identifying opportunities for future development.

4. Background and Literature Review

4.1 The IoT Ecosystem and Interoperability Problem

The Internet of Things has seen rapid growth over the past decade, with smart home devices becoming increasingly common in domestic environments. However, this growth has not been accompanied by a unified approach to device communication. Instead, the industry has evolved around a fragmented collection of competing wireless protocols, each with different design goals, frequency bands, network topologies, and application layers. The result is an ecosystem where devices from different manufacturers frequently cannot communicate with each other without proprietary hubs, cloud intermediaries, or significant integration effort [1].

The most widely deployed protocols in the smart home space each occupy a different position in this landscape. Zigbee is a low-power mesh networking protocol operating on the 2.4 GHz band, based on the IEEE 802.15.4 radio standard. It has broad device support and good range characteristics in mesh configurations, but interoperability between Zigbee devices from different manufacturers has historically been inconsistent due to variations in application profiles [1]. Z-Wave is a similar mesh protocol operating at sub-GHz frequencies, which gives it better wall penetration at the cost of lower data rates. While Z-Wave has stronger cross-manufacturer interoperability guarantees than Zigbee, it remains a proprietary standard. Wi-Fi offers high bandwidth and requires no additional hub hardware, but its power consumption makes it unsuitable for battery-operated devices, and each device typically connects independently to a manufacturer's cloud service. Thread is a newer IP-based mesh protocol also built on IEEE 802.15.4, designed to be fully IPv6-native - but Thread itself defines no application layer, meaning devices still required a common application protocol to achieve meaningful interoperability [1].

The practical consequence of this fragmentation is that most smart home deployments require either a single manufacturer's ecosystem, or a proprietary hub acting as a translator between protocols. Neither approach is satisfactory. Manufacturer lock-in restricts user choice and creates a dependency on continued cloud service provision, while proprietary translation hubs add cost and complexity and still typically require cloud connectivity to function. The industry recognised this problem early - in 2016 the Zigbee Alliance and the Thread Group announced a collaboration specifically aimed at reducing fragmentation, describing it as "a big step towards reducing fragmentation in the industry" [2]. However, these efforts fell short of delivering true cross-ecosystem interoperability, and it was not until the release of Matter in 2022 that a credible open standard emerged to address the problem at the application layer across Wi-Fi, Thread, and Ethernet simultaneously.

4.2 The Matter Protocol

4.2.1 Overview and Governance

Matter is an open-source, IP-based smart home connectivity standard developed under the Connectivity Standards Alliance (CSA), an industry body whose members include Apple, Google, Amazon, Samsung, and many other major technology companies [3]. The project began in 2019 under the name Project CHIP (Connected Home over IP), with the goal of creating a single, unified application-layer protocol that would allow smart home devices to communicate regardless of manufacturer or underlying radio technology. Matter 1.0 was formally released in October 2022, marking the first production-ready version of the standard [4].

The core goals of Matter are interoperability, security, and local control. Interoperability is achieved by defining a common device model and application layer that sits on top of

existing IP-based network technologies - devices can communicate over Wi-Fi, Thread, or Ethernet while all speaking the same application protocol. Security is built into the standard from the ground up, with each device required to hold a Device Attestation Certificate (DAC) issued by a CSA-authorised vendor, ensuring that only genuine, certified devices can be commissioned onto a Matter network. Local control means that all communication between a Matter controller and its devices happens entirely on the local network, with no cloud dependency at runtime [4].

Because Matter is an open standard governed by the CSA rather than a single company, any manufacturer can implement it without licensing fees. This is a significant shift from the proprietary hub model that has dominated the smart home industry, and it is the foundation on which PiHub is built [3].

4.2.2 Network Layer and Transport

Matter operates at the application layer of the network stack, sitting on top of standard IP-based transport protocols rather than defining its own. At the network and transport layers, Matter uses IPv6 with either UDP or TCP as the transport, depending on message type. This design means that any IP-capable network technology - Wi-Fi, Thread, or Ethernet - can carry Matter traffic without modification, which is central to its interoperability goals [4].

Service discovery is handled using Multicast DNS (mDNS), a zero-configuration protocol that allows devices to announce and find services on a local network without requiring a dedicated DNS server. When a Matter device joins a network, it broadcasts its presence via mDNS so that controllers on the same subnet can discover it automatically. In PiHub, this is how python-matter-server detects commissioned nodes on the local Wi-Fi network.

Matter defines three distinct roles within its architecture. A Commissionee is a device that has not yet joined a Matter network and is waiting to be set up. A Commissioner is responsible for onboarding new devices onto the network - in PiHub this role is fulfilled by python-matter-server during the commissioning process. Once a device has been commissioned it becomes an Accessory, and ongoing communication with it is handled by a Controller. In many deployments, including PiHub, the Commissioner and Controller roles are fulfilled by the same software instance [5].

All commissioned devices and controllers belong to a shared logical network called a Fabric. A Fabric defines the trust boundary of a Matter deployment - devices on the same Fabric share a root certificate authority and can communicate securely with each other. A single device can belong to multiple Fabrics simultaneously, which is how a device can be controlled by both, for example, Apple Home and a custom hub like PiHub without conflict.

4.2.3 Commissioning

Commissioning is the process by which a new device is securely onboarded onto a Matter Fabric. It is a one-time setup procedure that establishes the trust relationship between a device and a controller, after which the device can be discovered and controlled on the local network without any further setup [5].

The process begins with discovery. For a device that has never been commissioned before, this is done over Bluetooth Low Energy (BLE). The device advertises its presence via BLE, and the Commissioner - in PiHub's case, python-matter-server - scans for and connects to it. To initiate commissioning, the user provides an onboarding payload, either by scanning a QR code printed on the device or by entering a manual pairing code. This payload contains a setup passcode and discriminator value that uniquely identify the device.

Once the Commissioner has the onboarding payload, it establishes a secure channel with the device using PASE (Password Authenticated Session Establishment), specifically the SPAKE2+ protocol. This uses the setup passcode as a shared secret to derive encryption keys, allowing the Commissioner and device to communicate securely without any prior trust relationship. Over this secure channel, the Commissioner performs device attestation - verifying that the device holds a valid Device Attestation Certificate (DAC) issued by a CSA-authorised vendor, confirming the device is genuine and certified [4].

After attestation passes, the Commissioner installs operational credentials onto the device in the form of a Node Operational Certificate (NOC). This certificate ties the device to the Fabric and assigns it a unique Node ID. From this point on, all communication between the controller and the device uses CASE (Certificate Authenticated Session Establishment) rather than PASE, with sessions secured using the operational certificates.

In PiHub, both ESP32-C3 nodes were commissioned individually using the default manual pairing code (34970112332) and python-matter-server's commissioning command. One practical issue encountered was that the Vodafone router used during development rejected the ESP32-C3's Wi-Fi authentication during commissioning. This was resolved by commissioning over an iPhone hotspot instead, which provided a more permissive 2.4 GHz network that the ESP32-C3 could reliably connect to.

4.2.4 Device Model and Clusters

The Matter device model defines a structured hierarchy for how a device exposes its functionality to a controller. Understanding this model is important for understanding how PiHub communicates with its ESP32-C3 nodes.

At the top level is a Node, which represents a single physical device on the Fabric - each ESP32-C3 in PiHub is a Node. A Node is divided into one or more Endpoints, where each Endpoint represents a distinct logical function of the device. Endpoint 0 is reserved on every Matter device as the Root Node endpoint, containing administrative clusters used for commissioning and device management. Additional endpoints carry the actual application functionality - for example, the DHT22 sensor node exposes a temperature measurement endpoint and a humidity measurement endpoint as separate logical units [4].

Each Endpoint contains one or more Clusters, which are the core building blocks of the Matter application layer. A Cluster groups together related functionality under a standardised interface. There are two types: server clusters, which are hosted on the device and expose data or accept commands, and client clusters, which are hosted on the controller and interact with server clusters on devices. Each Cluster contains Attributes, which are the data values that can be read or subscribed to, Commands, which are actions that can be sent to the device, and Events, which are notifications the device can emit when something noteworthy occurs.

A concrete example is the OnOff cluster (cluster ID 0x0006), which is one of the simplest in the Matter specification. It contains a single boolean Attribute called OnOff that represents whether a device is on or off, and two Commands - On and Off - that a controller can send to change that state. The RGB LED node in PiHub implements the OnOff cluster on its light endpoint, allowing the controller to toggle the LED on and off with a standardised command that any Matter controller would understand.

The ColorControl cluster (cluster ID 0x0300) is more complex and is also implemented on the LED node. It supports multiple colour modes, but PiHub uses the XY colour mode, which represents colours using CIE 1931 XY chromaticity coordinates. The cluster exposes CurrentX and CurrentY attributes that hold the current colour, and a MoveToColor command that the controller sends to change it. Using XY colour space rather than hue and saturation gives more precise and consistent colour reproduction across different LED hardware.

4.2.5 Security Model

Security is a first-class concern in the Matter specification, and the protocol includes several layers of security that operate at different stages of a device's lifecycle. For a self-hosted hub like PiHub, understanding this model is important because it explains why the system can be trusted without relying on a cloud authority to validate devices at runtime.

The foundation of Matter's security model is device attestation. Every Matter-certified device is required to hold a Device Attestation Certificate (DAC), which is a unique X.509 certificate provisioned into the device's secure storage during manufacturing. The DAC is signed by the device vendor's Product Attestation Intermediate (PAI) certificate, which in turn is signed by a Product Attestation Authority (PAA) - a root certificate authority that has been approved by the CSA. During commissioning, the Commissioner verifies this certificate chain to confirm that the device is a genuine, CSA-certified product and not a counterfeit or rogue device attempting to join the Fabric [4].

Alongside the DAC, each device also carries a Certificate Declaration (CD), which is a CSA-issued document that declares the device's vendor ID, product ID, and the clusters it supports. The Commissioner validates the CD during attestation as an additional check that the device is what it claims to be.

Once commissioning is complete, all subsequent communication between the controller and the device uses CASE, as described in Section 4.2.3. Messages are encrypted using AES-CCM, meaning that even on a shared local network, Matter traffic cannot be read or injected by other devices on the same subnet [5].

For a self-hosted hub like PiHub, this security model has an important practical implication. Because all trust is rooted in the device's DAC and the Fabric's shared certificate authority rather than in a cloud service, the system remains fully secure even with no internet connectivity. The hub validates devices locally during commissioning, and all subsequent sessions are authenticated using certificates that are stored on the device and the hub. There is no token refresh, no cloud login, and no dependency on a manufacturer's servers remaining online.

4.3 Microcontroller Platform: ESP32-C3

4.3.1 Hardware Specification

The ESP32-C3-DevKitM-1 is a development board produced by Espressif Systems, built around the ESP32-C3 system-on-chip (SoC). While it belongs to the broader ESP32 family, a key distinction of the ESP32-C3 over the original ESP32 is its official support for the Matter protocol via Espressif's Esp-matter SDK. The original ESP32 does not have Matter support, making the ESP32-C3 the appropriate choice for a project built around the Matter standard [6].

The ESP32-C3 SoC is based on a 32-bit RISC-V single-core processor running at up to 160 MHz. It includes 400 KB of SRAM and 384 KB of ROM on-chip, with 4 MB of external flash storage on the DevKitM-1 board for firmware and NVS (Non-Volatile Storage) partition data. The wireless subsystem supports IEEE 802.11 b/g/n Wi-Fi on the 2.4 GHz band and Bluetooth 5.0 with BLE support. Both are required for Matter - BLE is used during the commissioning process for initial device discovery, and Wi-Fi provides the IP network over which all operational Matter traffic runs [7].

The DevKitM-1 board exposes a set of GPIO pins used to interface with external peripherals. On the sensor node, a GPIO pin communicates with the DHT22 sensor via its single-wire protocol. On the LED node, GPIO 8 is connected to the onboard WS2812 RGB LED and is driven using the ESP32-C3's RMT (Remote Control Transceiver) peripheral, which provides the precise timing required by the WS2812 protocol. The board is powered and programmed via USB-C, with serial output for debugging available on the same connection [7].

4.3.2 ESP-IDF and Esp-matter Framework

ESP-IDF (Espressif IoT Development Framework) is the official development framework for Espressif microcontrollers, including the ESP32-C3. Unlike hobbyist environments such as the Arduino IDE, ESP-IDF is a professional-grade embedded development framework built on top of FreeRTOS, providing full access to the hardware peripherals, a robust task scheduling system, and a comprehensive set of libraries for networking, storage, and security. Development is done in C and C++, and the build system is based on CMake, which gives the developer fine-grained control over project structure, component dependencies, and compile-time configuration [6].

One of the key advantages of ESP-IDF over Arduino-style development is the sdkconfig system. Rather than hardcoding settings in source files, ESP-IDF uses a Kconfig-based configuration system where compile-time options - such as Wi-Fi credentials, partition table layout, logging verbosity, and peripheral settings - are managed through a structured configuration file. For PiHub, a sdkconfig.defaults and sdkconfig.defaults.esp32c3 file was used to define the target-specific settings for the ESP32-C3, keeping the configuration separate from the application code and making the build reproducible across environments.

```
#disable BT connection reattempt
CONFIG_BT_NIMBLE_ENABLE_CONN_REATTEMPT=n

#enable lwip ipv6 autoconfig
CONFIG_LWIP_IPV6_AUTOCONFIG=y

# Use a custom partition table
CONFIG_PARTITION_TABLE_CUSTOM=y
CONFIG_PARTITION_TABLE_FILENAME="partitions.csv"
CONFIG_PARTITION_TABLE_OFFSET=0xC000
```

Figure 4-1: sdkconfig.defaults configuration file for ESP32-C3 DHT22 Node

The Esp-matter SDK is Espressif's open-source implementation of the Matter protocol for ESP32 devices, built on top of the connectedhomeip stack - the reference implementation of Matter maintained by the CSA on GitHub. The Esp-matter SDK wraps connectedhomeip and provides a set of higher-level abstractions specifically tailored to ESP32 hardware, including factory NVS (Non-Volatile Storage) partitioning for storing device credentials and commissioning data, pre-built cluster definitions that map directly to Matter's device model, and driver integration for common peripherals. For PiHub, the release/v1.5 branch of Esp-matter was used [6].

A key concept in Esp-matter is the use of cluster attribute callbacks. When a Matter controller sends a command or writes an attribute - such as turning the LED on or changing its colour - Esp-matter invokes a registered C++ callback function in the application code. This is where the application logic lives: the callback reads the new attribute value and drives the hardware accordingly. This event-driven model keeps the Matter stack and the application code cleanly separated, which is important for maintainability in a project of this complexity.

4.3.3 Peripheral Subsystems

Two peripheral subsystems on the ESP32-C3 are directly relevant to PiHub: the GPIO peripheral used for DHT22 sensor communication, and the RMT (Remote Control Transceiver) peripheral used for driving the WS2812 RGB LED.

The DHT22 communicates over a single-wire protocol where both the microcontroller and the sensor share the same data line, with the direction of communication changing throughout the exchange. The ESP32-C3 initiates a reading by pulling the line low for a defined period, after which the DHT22 responds with a stream of bits encoded as timed high pulses. This is handled directly through GPIO, with the firmware toggling the pin direction between output and input at the appropriate points in the protocol. A 10,000 Ohm pull-up resistor on the data line keeps it high when neither side is actively driving it.

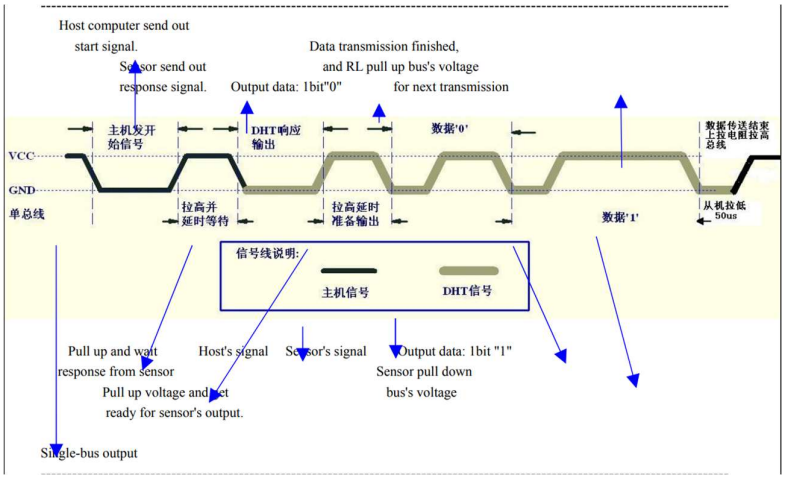


Figure 4-2: Aosong DHT22 (AM2302) timing diagram

The WS2812 LED requires a more precise signalling approach than the DHT22. It uses a single-wire protocol where each bit is encoded as a high pulse of a specific duration. A logical 0 is represented by a short high pulse (T0H) followed by a longer low (T0L), while a logical 1 is represented by a longer high pulse (T1H) followed by a shorter low (T1L), as shown in Figure 4-3. These timing tolerances are on the order of hundreds of nanoseconds, and a reset code (Treset) is used to signal the end of a data transmission. Simply toggling a GPIO pin in software is not reliable enough to meet these tolerances, as the CPU can be interrupted mid-sequence by other tasks, throwing the timing off.

The RMT (Remote Control Transceiver) peripheral is used instead of PWM because, unlike PWM, which is designed for continuous repeating waveforms, RMT can transmit a precise one-shot sequence of pulses autonomously without any CPU involvement. This guarantees the timing accuracy the WS2812 requires. In PiHub, the Esp-matter LED driver uses the RMT peripheral to send 24-bit GRB colour data to the WS2812 on GPIO 8.

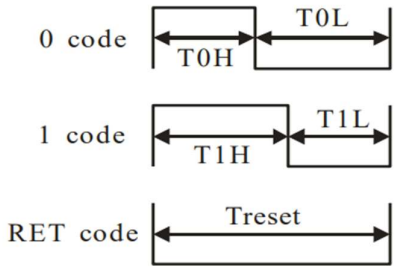


Figure 4-3: WS2812B bit encoding timing diagram showing T0H/T1H pulse durations

4.4 Sensors and Actuators

4.4.1 DHT22 Temperature and Humidity Sensor

The DHT22 is a low-cost digital temperature and humidity sensor commonly used in embedded systems. It measures temperature in the range of -40°C to +80°C with an accuracy of +/-0.5°C, and relative humidity in the range of 0-100% RH with an accuracy of +/-2-5% RH. Resolution is 0.1°C and 0.1% RH respectively. The sensor outputs readings over a single-wire protocol, communicating with the host microcontroller via a series of timed pulses on a shared data line, as described in Figure 4-2 in Section 4.3.3 [8].

In the Matter device model, the DHT22 sensor node exposes two separate server clusters. Temperature readings are mapped to the TemperatureMeasurement cluster (cluster ID 0x0402), which exposes a MeasuredValue attribute holding the current temperature as a signed 16-bit integer in units of 0.01°C. Humidity readings are mapped to the RelativeHumidityMeasurement cluster (cluster ID 0x0405), which exposes a MeasuredValue attribute holding the current humidity as an unsigned 16-bit integer in units of 0.01% RH. The firmware reads the sensor at a fixed interval and updates these attributes, which python-matter-server then subscribes to and forwards to the FastAPI backend [4].

4.4.2 WS2812 Addressable RGB LED

The WS2812 is an addressable RGB LED with an integrated driver IC, meaning each LED contains its own control circuitry and can be driven directly from a single data line without external driver components. It represents colour using 24-bit GRB encoding - 8 bits each for green, red, and blue - giving 16.7 million possible colours. The signalling protocol and timing constraints were covered in Figure 4-3 in Section 4.3.3 [9].

In the Matter device model, the WS2812 node exposes the ColorControl cluster using XY colour mode. Rather than working in RGB or HSV directly, Matter's XY mode represents colours using CIE 1931 chromaticity coordinates, where any visible colour can be described as an X and Y value between 0 and 1. The frontend converts a user-selected hex colour to CIE XY coordinates before sending a MoveToColor command to the node. This approach is more colour-accurate than hue and saturation mode and maps more cleanly to the underlying physics of how colour is perceived. The node also implements the LevelControl cluster for brightness control and the OnOff cluster for toggling the LED on and off.

4.5 Backend and Database Technologies

4.5.1 FastAPI

FastAPI is a modern Python web framework built on the ASGI (Asynchronous Server Gateway Interface) standard, meaning it is designed from the ground up to handle asynchronous operations natively using Python's async/await syntax. It was chosen as the backend framework for PiHub for several reasons: its native async support makes it well suited to managing a persistent WebSocket connection to python-matter-server alongside handling HTTP requests from the frontend, its automatic OpenAPI documentation generation and Swagger UI endpoint testing is useful during development, and its dependency injection system keeps route handlers clean and testable [10].

In PiHub, FastAPI sits at the centre of the software stack. It maintains a persistent WebSocket connection to python-matter-server, receiving real-time attribute updates from the ESP32-C3 nodes and caching the latest values in memory for the frontend to poll. It also exposes REST endpoints that the React frontend calls to read sensor history from the database and send control commands to the LED node.

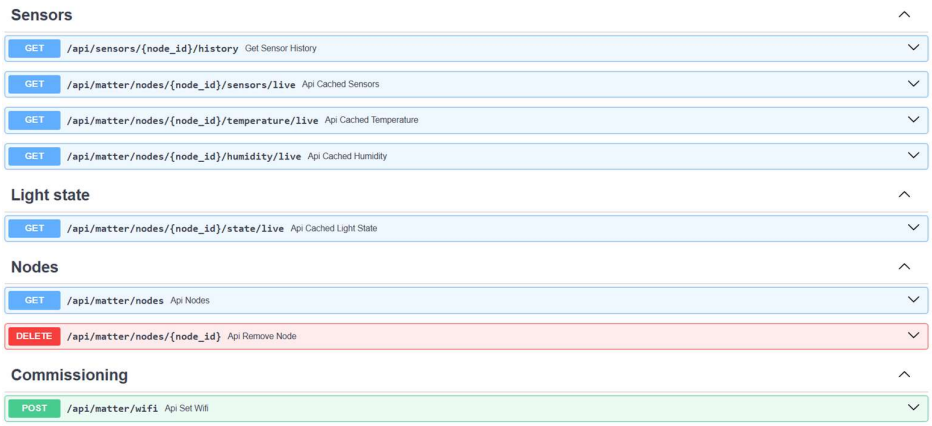


Figure 4-4: Swagger UI project endpoints sample.

4.5.2 PostgreSQL

PostgreSQL is a production-grade open-source relational database known for its ACID compliance, robust concurrency handling, and broad ecosystem support. PiHub initially used SQLite as its database during early development due to its simplicity and zero-configuration setup. However, SQLite was replaced with PostgreSQL as the project matured, primarily because SQLite does not handle concurrent writes well - an issue that becomes relevant when the Matter WebSocket listener and the HTTP request handlers are writing to the database simultaneously in an async context. PostgreSQL also has significantly better support in Docker environments, with an official image that integrates cleanly into the Compose stack [11].

The schema consists of two tables. The devices table stores a record for each commissioned Matter node, holding a unique node_id and a human-readable name. The sensor_readings table stores all historical sensor data, with each row recording a node_id, a sensor_type (either "temperature_c" or "humidity_rh"), a float value, and a timezone-aware timestamp that defaults to the current UTC time at the point of insertion.

```
class DeviceDB(Base):
    __tablename__ = "devices"

    id: Mapped[int] = mapped_column(primary_key=True, index=True)
    node_id: Mapped[int] = mapped_column(Integer, unique=True, index=True, nullable=False)
    name: Mapped[str] = mapped_column(String, nullable=False)

class SensorReadingDB(Base):
    __tablename__ = "sensor_readings"

    id: Mapped[int] = mapped_column(primary_key=True, index=True)
    node_id: Mapped[int] = mapped_column(Integer, index=True, nullable=False)
    sensor_type: Mapped[str] = mapped_column(String, nullable=False)  # "temperature_c" | "humidity_rh"
    value: Mapped[float] = mapped_column(Float, nullable=False)
    timestamp: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        default=lambda: datetime.now(timezone.utc),
        nullable=False,
    )
```

Figure 4-5: SQLAlchemy ORM model definitions.

4.5.3 SQLAlchemy

SQLAlchemy is a Python SQL toolkit and ORM (Object Relational Mapper) that provides a high-level interface for interacting with relational databases. Rather than writing raw SQL queries, the application defines database tables as Python classes and interacts with them through SQLAlchemy's query API, as seen in the model definitions in Figure 4-5.

PiHub uses SQLAlchemy's synchronous Core rather than its async variant. This is because the async SQLAlchemy API adds considerable complexity around session management and connection pooling, and the performance benefit was not justified given the relatively low write frequency of sensor data. To avoid blocking the FastAPI async event loop when performing database operations, all SQLAlchemy calls are wrapped in FastAPI's run_in_executor, which offloads the synchronous database call to a threadpool and awaits the result without blocking other async tasks.

4.6 Frontend Technologies

4.6.1 React and Vite

React is a JavaScript library for building user interfaces through a component-based model, where the UI is broken down into reusable, self-contained components that manage their own state and rendering. Vite is a modern frontend build tool that provides a fast development server with hot module replacement (HMR), meaning changes to source files are reflected in the browser instantly without a full page reload. For production, Vite compiles the React application into a set of static HTML, CSS, and JavaScript files that are served by Nginx in the PiHub Docker stack.

React was chosen over alternatives like Next.js because server-side rendering (SSR), which is Next.js's primary advantage, is unnecessary for PiHub's use case. The dashboard is a single-page kiosk application running on a fixed device - there are no SEO requirements, no dynamic routing that benefits from SSR, and no need for the additional complexity that Next.js introduces. React with Vite gives a simpler, lighter setup that produces a static build output well suited to being served directly from Nginx.

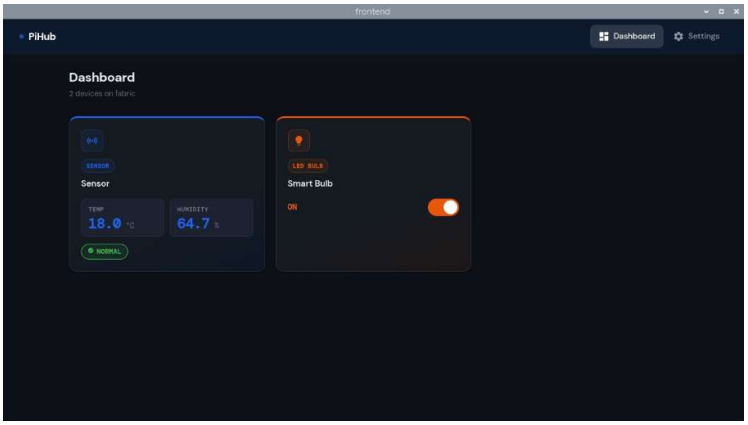


Figure 4-6: PiHub main dashboard.

4.6.2 Touch and Kiosk UI Considerations

The PiHub frontend is designed to run in Chromium kiosk mode on a Raspberry Pi touchscreen, which introduces a set of design constraints that differ from a standard desktop web application. Kiosk mode launches Chromium in full screen with no browser chrome, no address bar, and no window controls, meaning the application UI is the only thing visible on the screen.

The most significant design constraint is the absence of a mouse. Standard web UI patterns rely heavily on hover states to indicate interactivity - buttons change colour on hover, tooltips appear on hover, and so on. On a touchscreen none of this applies, as touch input jumps directly from no contact to a press with no hover state in between. To handle this, all :hover CSS rules were replaced with :active pseudo-classes throughout the frontend, so interactive elements provide visual feedback on touch press rather than on cursor proximity.

The CSS custom property token system was also an important part of the frontend architecture. Rather than hardcoding colour values throughout the stylesheet, all UI colours are defined as CSS custom properties (variables) at the root level - for example --ui-primary, --ui-background, and --ui-accent. This means the entire colour scheme of the dashboard can be changed by updating a small set of variables in one place, which was particularly useful for implementing the customisable tile accent colours that are decoupled from the main UI chrome tokens.

4.7 Containerisation and DevOps

4.7.1 Docker and Docker Compose

Docker is a containerisation platform that packages an application and all its dependencies into a portable, self-contained unit called a container. Docker Compose extends this by allowing multiple containers to be defined, configured, and started together as a single stack using a YAML file. For PiHub, Docker Compose is used to manage all services on the Raspberry Pi, making the deployment reproducible and straightforward to update [12].

PiHub uses a two-stack Docker Compose architecture rather than a single stack. This separation exists because of a fundamental constraint with python-matter-server. Matter relies on mDNS for device discovery, which uses IPv4/IPv6 multicast on the local network. It also requires access to the host machine's Bluetooth adapter and DBus socket for commissioning. None of these work correctly when a container is on a Docker bridge network, because bridge networking isolates the container from the host's network interfaces. For this reason, python-matter-server runs in its own separate Compose stack with network_mode: host, giving it direct access to the host network stack, Bluetooth adapter, and DBus.

The main application stack - containing the FastAPI backend, PostgreSQL database, and Nginx frontend server - runs on a standard Docker bridge network. Because the backend needs to communicate with python-matter-server, which is running on the host network rather than the bridge network, it connects to it using the special host.docker.internal hostname. This is a DNS name that Docker resolves to the host machine's IP address from within a bridge-networked container, allowing the backend to reach python-matter-server's WebSocket without the two services needing to be on the same network. The MATTER_WS_URL environment variable in the backend container is set to this address at deploy time. The full Docker stack is visible in my architecture diagram in figure 5-1 in section 5.1.

4.7.2 CI/CD with GitHub Actions and GHCR

Continuous Integration and Continuous Deployment (CI/CD) is a software development practice where code changes are automatically built, tested, and deployed through a pipeline rather than manually. PiHub uses GitHub Actions to implement this pipeline, with Docker images published to the GitHub Container Registry (GHCR) [13].

Every push to the main branch triggers the workflow. The pipeline is structured as three parallel jobs that run simultaneously - a pytest job that runs the FastAPI backend test suite, and a Vitest job that runs the React frontend test suite. Both test jobs must pass before a third job is allowed to proceed, which builds multi-architecture Docker images for both the backend and frontend and pushes them to GHCR with a version tag. This ensures that a broken test suite can never result in a bad image being published.

On the Raspberry Pi, deployment is managed through a separate repository that holds only the Docker Compose files and environment configuration. To deploy a new version, the Pi pulls the latest images from GHCR and restarts the stack with two commands - docker

compose pull followed by docker compose up -d. This keeps the deployment process simple and means the Pi never needs access to the source code, only to the published images.

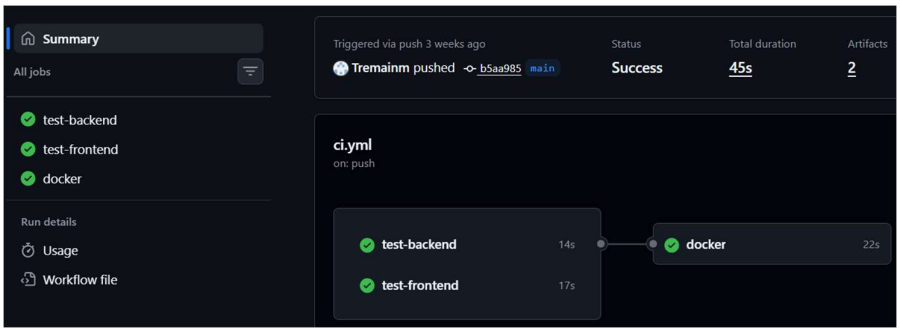


Figure 4-7: CI/CD workflow showing parallel test jobs gating the Docker build step.

4.8 TinyML and On-Device Inference

4.8.1 TensorFlow and TensorFlow Lite Micro

TensorFlow is an open-source machine learning framework developed by Google that provides tools for building, training, and evaluating neural network models. Models are typically defined and trained using Keras, TensorFlow's high-level API, which simplifies the process of defining layer architectures, compiling models, and running training loops. TensorFlow Lite is a separate runtime designed for deploying trained models on resource-constrained devices, and TensorFlow Lite Micro (TFLM) is a further stripped-down version of that runtime specifically targeting microcontrollers with no operating system, no dynamic memory allocation, and very limited flash and RAM [14].

The reason TFLM is necessary for PiHub rather than running inference on the Raspberry Pi is that the goal was to perform classification directly on the ESP32-C3 at the point of sensing, without any round-trip to the hub. The ESP32-C3 has 400 KB of SRAM and 4 MB of flash, which is enough for a small, quantised model but nowhere near enough to run a standard TensorFlow runtime. TFLM addresses this by providing a minimal inference engine with a fixed memory arena and no dynamic allocation at runtime.

Int8 quantisation is the process of converting a trained model's weights and activations from 32-bit floating point values down to 8-bit integers. This reduces the model size significantly and makes inference faster and more memory-efficient on hardware that lacks a floating point unit, which is the case for the ESP32-C3. The accuracy trade-off is minimal for a simple classification task like PiHub's, making it a practical choice for embedded deployment.

4.8.2 Context Classification Approaches

The initial approach to on-device intelligence in PiHub was anomaly detection using an autoencoder - a type of neural network trained to reconstruct its input, where a high reconstruction error indicates that the input is unusual. The idea was that the model would learn what "normal" sensor readings look like and flag deviations. In practice this approach had a significant limitation: it could tell that something was different, but not what was different. A single reconstruction error threshold cannot distinguish between a room heating up and a window being opened, even though these are meaningfully different contexts that might warrant different responses. The approach was abandoned in favour of supervised classification.

A supervised multiclass classifier takes a set of input features and outputs a predicted class from a fixed set of labels. Several approaches were considered for PiHub. k-Nearest

Neighbours (k-NN) classifies an input by finding the k most similar training examples and taking a majority vote - simple and interpretable but requires storing the entire training dataset in memory at inference time, which is impractical on a microcontroller. Decision trees and random forests offer good accuracy and are easy to reason about but are difficult to quantise and deploy on TFLM without additional conversion tooling.

A feed-forward neural network, also known as a Multi-Layer Perceptron (MLP), was chosen as the most practical option for this use case. With only two input features - temperature and humidity - the model can be kept very small, with a low parameter count that fits comfortably within the ESP32-C3's memory constraints after int8 quantisation. TFLM has mature support for dense layer inference, and the three output classes - HEATING_ON, NORMAL, and WINDOW_OPEN - map directly to the SoftMax output of a classification network. The structured, well-defined nature of the label space made supervised classification a natural fit, since the boundaries between classes are determined by known physical conditions rather than statistical anomalies.

5. System Architecture

5.1 High-Level Architecture Overview

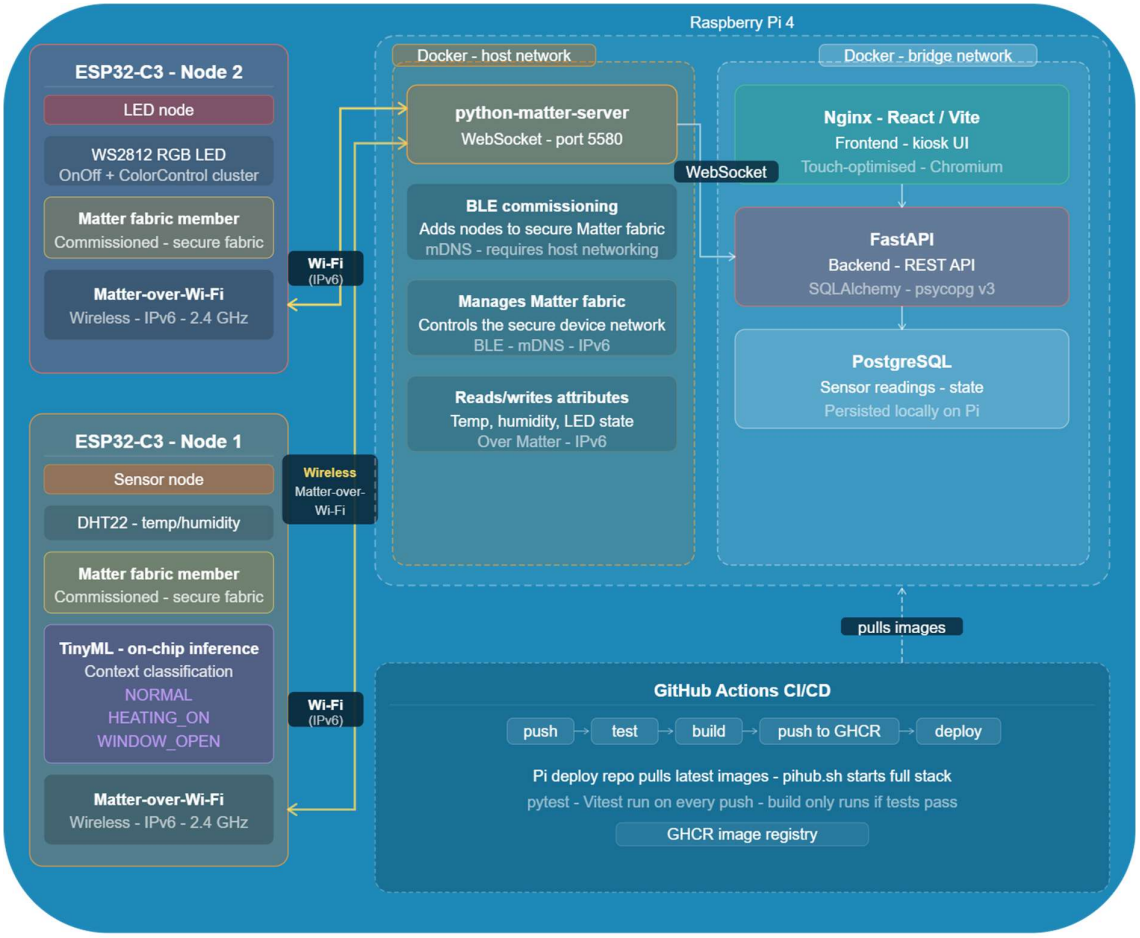


Figure 5-1: PiHub Full System Architecture Diagram.

5.2 Hardware Architecture

5.2.1 Central Hub: Raspberry Pi 4

The Raspberry Pi 4 acts as the central hub of the PiHub system, hosting all hub software and serving as the Matter controller for the two ESP32-C3 nodes. The specific model used is the Raspberry Pi 4 Model B with 8 GB of RAM, featuring a quad-core ARM Cortex-A72 processor running at 1.8 GHz, onboard Wi-Fi, Bluetooth 5.0, and a Gigabit Ethernet port. The operating system is Debian Trixie, running the labwc window compositor on Wayland rather than a traditional desktop environment, keeping the system lightweight and suited to kiosk use.

The Pi serves three distinct roles in the system. As the Docker host, it runs both Compose stacks - the host-networked python-matter-server stack and the bridge-networked application stack containing FastAPI, PostgreSQL, and Nginx. As the Matter controller, it manages the secure Fabric that both ESP32-C3 nodes are commissioned onto, handling attribute subscriptions and sending control commands via python-matter-server. As the kiosk display server, it drives a touchscreen display running Chromium in kiosk mode, with the React dashboard set to launch automatically on startup via the labwc autostart configuration.

5.2.2 Sensor Node: ESP32-C3 + DHT22

The sensor node consists of an ESP32-C3-DevKitM-1 development board connected to a DFRobot DHT22 temperature and humidity sensor module. The DFRobot module integrates the pull-up resistor and decoupling capacitor required by the DHT22 onto the module PCB itself, simplifying the wiring to just three connections - VCC to the 3.3V pin on the ESP32-C3, GND to ground, and the data line to GPIO2, as shown in Figure 5-2. The board is powered via its micro-USB port for development and powered by a Li-Po 18650 battery & TP4056 charger module for production.

The sensor is polled at a fixed interval by the firmware, with the temperature and humidity readings mapped to the TemperatureMeasurement and RelativeHumidityMeasurement Matter clusters respectively, as described in Section 4.4.1. The node communicates with python-matter-server over Matter-over-Wi-Fi on the 2.4 GHz band using IPv6.

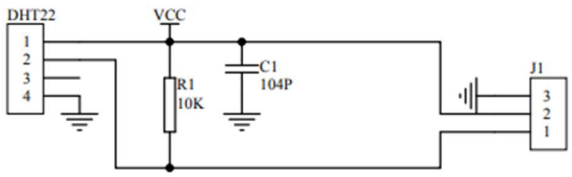


Figure 5-2: DFRobot DHT22 wiring schematic [15].

5.2.3 Actuator Node: ESP32-C3 + WS2812 LED

The actuator node is a standalone ESP32-C3-DevKitM-1 development board. Unlike the sensor node, it requires no external components - the WS2812 RGB LED is built directly onto the DevKitM-1 board and is connected internally to GPIO8, as shown in the pin layout in Figure 5-3 and the schematic wiring diagram is shown in Figure 5-4. The board is powered identically to the Sensor Node.

The LED is driven by the RMT peripheral as described in Section 4.3.3, with the firmware implementing the OnOff, LevelControl, and ColorControl Matter clusters to support toggling, brightness control, and full RGB colour control from the dashboard. The node communicates with python-matter-server over Matter-over-Wi-Fi on the 2.4 GHz band using IPv6.

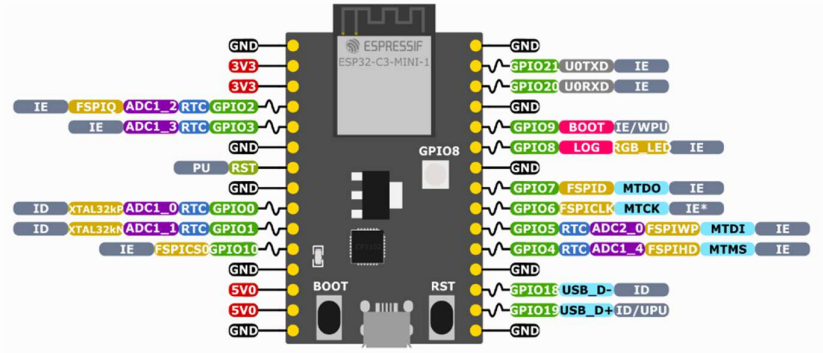


Figure 5-3: ESP32-C3-DevKitM-1 Pin Layout [16].

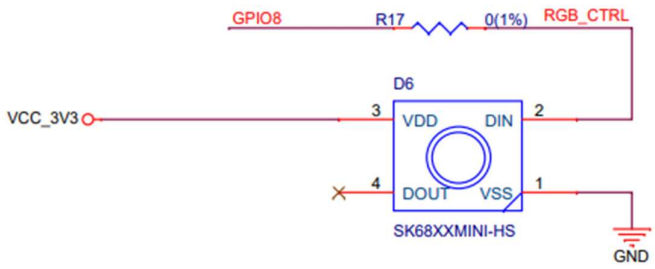


Figure 5-4: WS2812 LED wiring schematic [17].

5.2.4 Network Topology

All devices in the PiHub system operate on the same local Wi-Fi network. The Raspberry Pi 4 connects via Wi-Fi, and both ESP32-C3 nodes connect over 2.4 GHz Wi-Fi. There is no cloud dependency at runtime - all communication between the nodes and the hub happens entirely on the local network, with python-matter-server discovering and communicating with the nodes via IPv6 multicast and mDNS.

mDNS is central to how Matter devices are discovered on the network. When a commissioned node comes online, it advertises its presence via mDNS on the local subnet, allowing python-matter-server to locate it and establish a CASE session without any manual configuration. This is also why python-matter-server must run with host networking in Docker, as discussed in Section 4.7.1 - mDNS multicast does not function correctly through a Docker bridge network.

5.3 Software Architecture

5.3.1 Backend Service (FastAPI)

The FastAPI backend is structured as a Python package under the app/ directory, containing main.py as the application entry point, matter_ws.py as the Matter WebSocket client module, and supporting modules for the database connection (database.py), ORM models (models.py), and request/response schemas (schemas.py). A separate tests/ directory contains the pytest test suite, organised into individual test files per endpoint group.

The application uses FastAPI's lifespan context manager for startup and shutdown logic. On startup, the database tables are created if they do not exist, the background Matter WebSocket listener is started, and after a two-second pause to allow the initial attribute cache to populate, sensor callbacks are registered for the commissioned sensor node. The two-second delay is intentional - without it, callbacks could be registered before the start_listening handshake completes, causing the first few attribute updates to be missed.

The API is organised into tagged route groups visible in the auto-generated OpenAPI docs: Health, Devices, Sensors, Light state, Nodes, Commissioning, and LED control. All sensor and light state read endpoints are synchronous and return data directly from the in-memory cache maintained by matter_ws.py, making them fast and non-blocking. LED control and commissioning endpoints are async, as they dispatch commands to python-matter-server over the WebSocket and await a response [10].

CORS middleware is configured to allow all origins, which is appropriate for a local network deployment where the frontend and backend are on the same host but may be accessed from different ports during development. Figure 4-4 shows an example of Swagger UI and my endpoints.

5.3.2 Matter WebSocket Client (python-matter-server)

The matter_ws.py module manages the persistent WebSocket connection to python-matter-server and is the single point of contact between the FastAPI backend and the Matter fabric. It is responsible for sending commands to the nodes and maintaining a live cache of their current attribute values.

On startup, the module sends a start_listening command to python-matter-server over WebSocket. This command returns the current state of all commissioned nodes as a list - an important detail, as the response is a list rather than a dict and must be iterated accordingly. The module parses this initial dump and populates an in-memory attribute cache keyed by node ID and attribute path. From that point on, the WebSocket connection remains open and the module processes incoming messages on a background asyncio task, updating the cache whenever an attribute_updated event is received.

Attribute paths follow the format "endpoint/cluster/attribute" - for example, "1/1026/0" is the MeasuredValue attribute on the TemperatureMeasurement cluster at endpoint 1, and "2/1029/1" is the MinMeasuredValue attribute on the humidity cluster at endpoint 2, which is used as the TinyML context class workaround described in Section 4.2.4. When an attribute_updated event arrives for a registered path, the module fires any callbacks registered for that path, which in main.py are the functions that persist readings to PostgreSQL via a threadpool executor [18].

Commands such as turn_on, turn_off, set_brightness, and set_color_xy are implemented as async functions that send a JSON-RPC style command message over the WebSocket and await the response. Each command targets a specific node ID, endpoint, cluster, and attribute or command ID.

```
84  async def _background_listener() -> None:
108         try:
109             async with websockets.connect(
110                 MATTER_WS_URL,
111                 ping_interval=20,
112                 ping_timeout=20,
113             ) as ws:
114                 _ws = ws
115                 logger.info("Connected to matter-server - sending start_listening")
116
117                 listen_id = uuid.uuid4().hex
118                 await ws.send(json.dumps({
119                     "message_id": listen_id,
120                     "command": "start_listening",
121                 }))
122
123                 async for raw in ws:
124                     try:
125                         msg = json.loads(raw)
126                     except json.JSONDecodeError:
127                         logger.warning("Received non-JSON message, skipping")
128                         continue
129
130                     msg_id = msg.get("message_id")
131
132                     # -- 1. Resolve a pending _ws_call() Future ------------
133                     # Any message whose ID is in _pending is a command
134                     # response. Resolve the Future so the awaiting coroutine
135                     # continues with the result.
136                     if msg_id and msg_id != listen_id and msg_id in _pending:
137                         future = _pending.pop(msg_id)
138                         if not future.done():
```

Figure 5-5: matter_ws.py background_listener.

5.3.3 Database Layer

The database layer consists of two SQLAlchemy ORM models mapped to PostgreSQL tables, defined in models.py. The devices table stores a record for each commissioned Matter node with a unique node_id integer and a human-readable name string. The sensor_readings table stores all historical sensor data, with each row recording a node_id, a

sensor_type string ("temperature_c", "humidity_rh", or "context"), a float value, and a timezone-aware UTC timestamp that defaults to the time of insertion.

All database access uses SQLAlchemy's synchronous Session. Because the FastAPI application is async, synchronous database calls are offloaded to a threadpool using asyncio's run_in_executor, which runs the blocking call in a separate thread and awaits the result without blocking the event loop. This pattern is used in the sensor and context persistence callbacks in main.py, where _persist_sensor_reading is a plain synchronous function called via run_in_executor each time an attribute update arrives [11].

HTTP request handlers that need database access receive a Session instance via FastAPI's Depends(get_db) dependency injection, which yields a short-lived session from the connection pool and guarantees it is closed after the request completes regardless of whether it raises an exception. The ORM model tables are shown in Figure 4-5.

5.3.4 Frontend Application

The React application is structured around a clear separation between pages, components, hooks, API modules, and context. App.jsx is the root component, wrapping the entire application in a ThemeProvider and a BrowserRouter, and defining four client-side routes: the Dashboard at /, the LED control page at /led/:node_id, the sensor history page at /sensors/:node_id, and the Settings page at /settings.

The API layer is split into three modules - sensors.js, matter.js, and devices.js - each exporting focused functions that call the FastAPI backend via a shared client.js fetch wrapper. The client handles base URL resolution from the VITE_API_URL environment variable. This means no component ever calls fetch directly - all API interaction goes through these typed module functions.

The Dashboard page polls the sensor live endpoint every four seconds using a useCallback-stabilised interval, updating a sensorData state map keyed by node ID. Device names are loaded once on mount from the /api/devices endpoint. Sensor and LED devices are split by filtering the device list against the SENSOR_NODE_IDS and LED_NODE_IDS constants defined in nodeId.config.js, keeping the component logic clean and the node IDs configured in one place.

LED state is managed entirely by the useLedState hook, which polls the light state cache endpoint every four seconds and exposes toggle, changeBrightness, and changeColor commands. A pendingRef flag suppresses cache poll overwrites while a command is in flight, and a userSetColorRef flag prevents the xyToHex round-trip from drifting the colour after the user has explicitly chosen one. Brightness changes send the current colour via setColorXY before calling setBrightness, which prevents the white flash caused by the firmware resetting colour state when brightness is updated.

Theme and accent colour state is managed by ThemeContext, which persists preferences to localStorage and applies them by setting a data-theme attribute on the html element and injecting a dynamically built style tag with CSS custom property overrides for the sensor and LED accent colour token families.

Helper logic is extracted into standalone modules: ColourHelpers.js contains the hexToXY and xyToHex colour conversion functions, and ChartHelpers.jsx contains the timestamp formatter, shared chart tooltip component, and chart card component used in the sensor history page. AppearanceSettings.jsx contains the theme controls extracted from Settings.jsx. Figure 4-6 shows the main dashboard screen running.

5.3.5 Docker Compose Architecture

PiHub uses two separate Docker Compose stacks that must be started in a specific order. The Matter-server stack defined in docker-compose.matter.yml starts first, and the main application stack defined in docker-compose.yml starts second, after python-matter-server is confirmed ready on port 5580 [12].

The matter-server stack contains a single service running the official python-matter-server Docker image. It is configured with network_mode: host so that it has direct access to the host's network interfaces, Bluetooth adapter, and DBus socket - all of which are required for Matter commissioning and mDNS device discovery. The DBus socket is mounted read-only from /run/dbus on the host. The Matter fabric data and credentials are persisted in a volume mounted at /data inside the container. The service is started with explicit flags specifying the storage path, primary network interface (wlan0), Bluetooth adapter index, log level, and PAA root certificate directory.

The main application stack contains three services on a shared bridge network called pihub-net. The db service runs PostgreSQL 17 Alpine with a named volume for data persistence and a healthcheck that prevents the backend from starting before the database is ready [11]. The backend service runs the FastAPI application image pulled from GHCR, configured via an .env file, and set with extra_hosts: host.docker.internal:host-gateway so that it can reach python-matter-server running on the host network. The frontend service runs the Nginx image that serves the compiled React application, exposed on port 80.

Startup and shutdown of both stacks is managed by pihub.sh, a bash script in the deployment repository that handles the ordering dependency between the two stacks. It starts the matter-server stack first, polls port 5580 with nc until python-matter-server is accepting connections, then starts the application stack and polls port 80 until Nginx is ready. The script also supports --pull to fetch the latest images from GHCR before starting, --logs to attach to the log stream after startup, and --down-all to stop both stacks cleanly. My architecture diagram in Figure 5-1 shows the Docker Compose two-stack structure.

5.4 Data Flow

The end-to-end data flow in PiHub can be broken into two paths: the sensor data ingestion path from the ESP32-C3 to the database, and the command path from the React frontend to the LED node.

For sensor data, the DHT22 firmware reads temperature and humidity values at a fixed interval and updates the corresponding Matter cluster attributes on the device. python-matter-server, which maintains a persistent subscription to all commissioned nodes, receives an attribute_updated event over the Matter fabric and forwards it to the FastAPI backend via the WebSocket connection on port 5580. The matter_ws.py module receives the event, updates the in-memory attribute cache with the new value, and fires the registered callback for that attribute path. The callback calls _persist_sensor_reading via run_in_executor, which writes the reading to the sensor_readings table in PostgreSQL. On the frontend, the Dashboard polls the /api/matter/nodes/{node_id}/sensors/live endpoint every four seconds. This endpoint reads directly from the in-memory cache rather than the database, so it always returns the most recent value without a database query. Historical data for the SensorHistory charts is fetched separately from the /api/sensors/{node_id}/history endpoint, which queries PostgreSQL.

For LED control, the user interacts with the LedControl page - adjusting the colour picker, moving the brightness slider, or tapping the toggle. The useLedState hook debounces these interactions and calls the appropriate API function in matter.js, which sends an HTTP POST request to the FastAPI backend. The backend's async route handler calls the corresponding

function in matter_ws.py, which sends a JSON-RPC command over the WebSocket to python-matter-server. python-matter-server translates this into a Matter command and sends it to the LED node over the Wi-Fi fabric. The node's Esp-matter cluster callback receives the command and drives the WS2812 LED hardware accordingly.

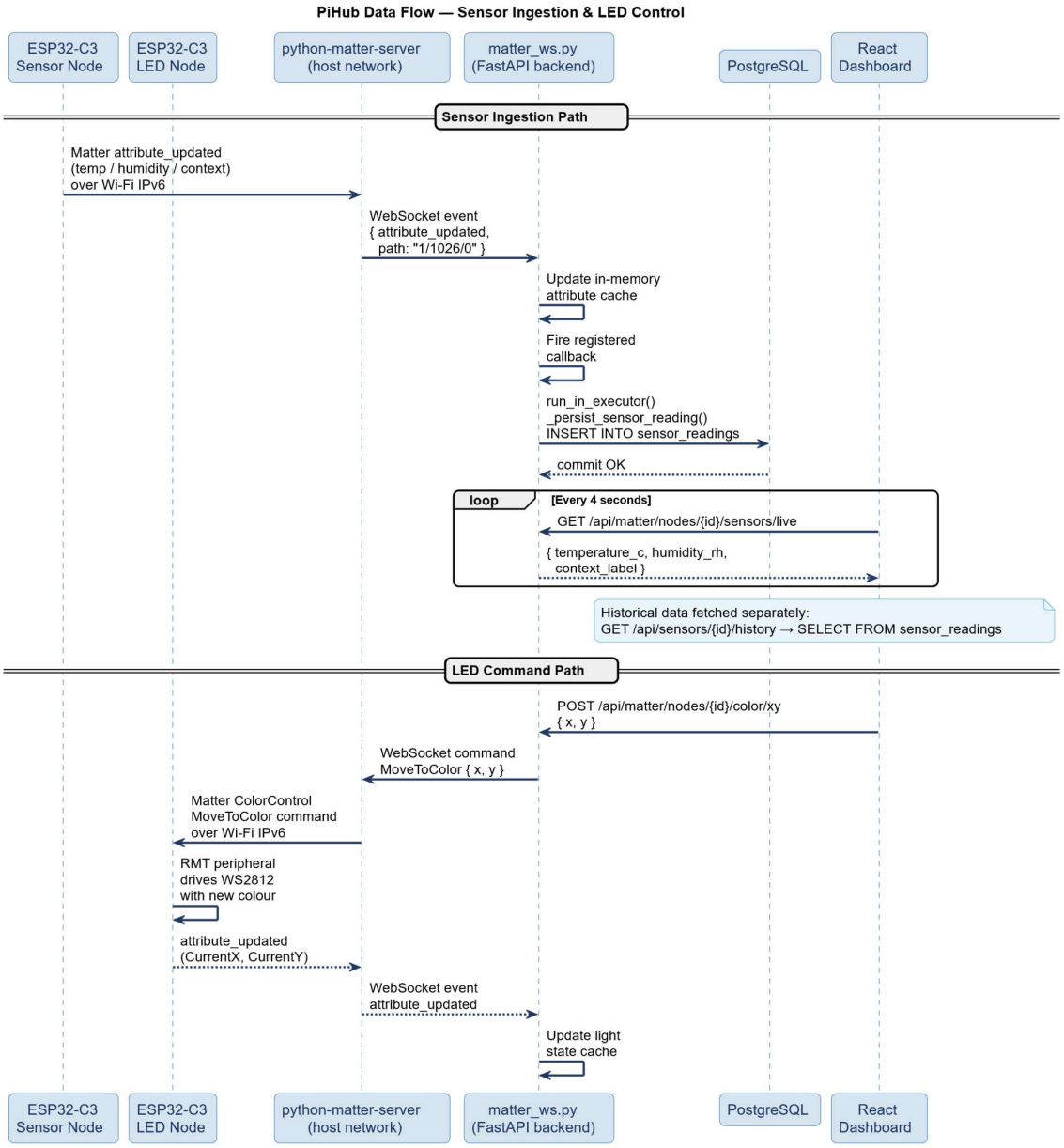


Figure 5-6: Sensor and LED control sequence diagram.

5.5 Security Architecture

PiHub's security model operates at several layers, from the Matter protocol itself down to the Docker network configuration.

At the device level, security is provided by Matter's device attestation and CASE session model, described in detail in Sections 4.2.3 and 4.2.5. Each ESP32-C3 node holds a Device

Attestation Certificate that was verified during commissioning, and all ongoing communication between python-matter-server and the nodes is encrypted using AES-CCM within an established CASE session. This means that even though all devices are on the same local Wi-Fi network, Matter traffic cannot be read or injected by other devices on the subnet [3].

At the network level, PiHub has no public-facing ports or cloud dependencies. The system is entirely local - there are no inbound connections from the internet, no manufacturer servers involved at runtime, and no data leaving the local network. The only ports exposed on the host are port 80 for the Nginx frontend and port 8000 for the FastAPI backend, both of which are accessible only from within the local network. python-matter-server runs on the host network for operational reasons but does not expose any service to the internet.

At the container level, Docker's bridge network provides isolation between services. The database is not exposed on any host port in the deployment Compose file - it is only reachable from within the pihub-net bridge network, meaning the PostgreSQL instance cannot be accessed directly from the local network, only by the backend container.

The main security limitation of the current implementation is the absence of authentication on the FastAPI backend. Any device on the local network can call the API endpoints, including LED control and commissioning. For the scope of this project, operating on a trusted home network, this was considered acceptable. A natural improvement would be to add token-based authentication to the API and restrict the Nginx frontend to HTTPS.

6. Project Plan

6.1 Project Management Methodology

PiHub was managed using an Agile-inspired iterative approach, with Jira used throughout the project as the primary task tracking tool. Work was organised into sprints, each representing a focused development phase - from initial research and setup through to firmware development, backend integration, TinyML, and final testing and documentation. Tickets were created as either Story type for new features and functionality, or Task type for refactoring, documentation, and maintenance work. Jira was used on a daily to weekly basis to create new tickets, move tasks through the in-progress and done columns, and keep track of what was outstanding at any given point.

The sprint structure visible in the Jira backlog reflects the natural progression of the project: early sprints covered research, project setup, and initial MQTT and HTTP exploration before the pivot to Matter; middle sprints covered firmware development, controller integration, and the backend and frontend build-out; and later sprints covered the TinyML workstream, automated testing, and project cleanup and wrap-up.

OneNote was used to track weekly progress with weekly logs. These include tasks or features achieved, issues or bugs encountered, and design decisions and solutions that were implemented. Weekly standups were also tracked to understand other students project progress and see any similarities.

6.2 Work Breakdown and Timeline

The project ran from October 2025 through April 2026, spanning approximately seven months. The table below summarises the major workstreams and their approximate timelines.

Phase	Description	Period
Research & Proposal	Project scoping, technology research, Matter protocol investigation, initial MQTT exploration, project proposal	Oct 2025
Setup & Initial Work	Pi setup, ESP32-C3 & DHT22 setup, Backend Setup, Basic React CRA setup	Oct – Nov 2025
MQTT & HTTP Setup	HTTP communication, MQTT between Pi & ESP32-C3, MQTT changed to Matter, Demo prep	Nov – Dec 2025
ESP-IDF Switch	Arduino IDE -> ESP-IDF, Matter controller over CHIP subprocess, research Matter light example	Jan – Feb 2026
Controller Integration	python-matter-server integration, Matter light firmware built, modify backend for sensor readings	Feb – Mar 2026
Project Cleanup	Containerise project, 3D print housings, refactor frontend code with Vite, switch to PostgreSQL	Mar – Apr 2026
TinyML Workstream	Context classifier design, train, quantise, integrate into DHT22 firmware	Apr 2026
Project Wrap-up	Setup Kiosk mode, study codebase, create poster, create report	Apr 2026

6.3 Milestones

The following table lists the key technical milestones achieved throughout the project.

First Matter device commissioned	First ESP32-C3 node successfully commissioned onto a Matter fabric using CHIP C++ subprocess tooling	Oct – Nov 2025
Sensor reading displayed on frontend	Sensor tile created to display incoming sensor reading. Reading updated by polling subprocess	Nov – Dec 2025
python-matter-server integration	WebSocket connection established, start_listening handshake working, attribute updates flowing to the backend cache	Feb – Mar 2026
LED Node commissioned & controlled	WS2812 LED node commissioned onto the fabric, full colour and brightness control working end-to-end from the React dashboard	Mar 2026
CI/CD Pipeline first green	GitHub Actions workflow passing all pytest and Vitest tests and building and pushing Docker images to GHCR	Apr 2026
TinyML running on DHT22 Node	Quantised context classifier running on-device, outputting class labels & being received on the Pi	Apr 2026
Full Stack Operational	All components running together on the Raspberry Pi - both nodes commissioned, sensor data flowing, LED control working, TinyML classifications surfacing in the dashboard	Apr 2026

7. Implementation

7.1 ESP32-C3 Firmware Development

7.1.1 Development Environment

The firmware for both ESP32-C3 nodes was developed on a Windows laptop using WSL2 (Windows Subsystem for Linux), with the VS Code ESP-IDF extension providing the build, flash, and monitor toolchain. The extension manages its own Python virtual environment and ESP-IDF installation, which means all idf.py commands must be run through the VS Code ESP-IDF extension's integrated terminal rather than a standard WSL shell. Running idf.py outside the extension shell breaks the virtual environment path and causes build failures that can be difficult to diagnose.

The Esp-matter SDK (release/v1.5 branch) was used as the Matter implementation layer on top of ESP-IDF. This branch was chosen for its stability and compatibility with the connectedhomeip version supported by python-matter-server 8.1.0. The SDK is cloned separately from ESP-IDF and the two are linked at build time through environment variables set by the extension [6].

Target-specific compile-time configuration is managed through a sdkconfig.defaults.esp32c3 file, which defines settings such as the partition table layout, NVS factory partition for Matter credentials, logging verbosity, and the RMT peripheral configuration for the LED node. This file is committed to the repository, making the build fully reproducible across machines without needing to manually configure the project each time. The sdkconfig file shown in Figure 4-1 shows the key settings used for the DHT22 sensor node [7].

One practical issue encountered during development was a conflict between VS Code's serial monitor and the pyserial data collection script used for the TinyML workstream. VS Code's monitor holds an exclusive lock on /dev/ttyUSB0, which prevents any other process from accessing the serial port. To release the port without closing VS Code entirely, the command sudo fuser -k /dev/ttyUSB0 was used to kill the process holding the lock, allowing pyserial to connect directly to the device for data collection.

7.1.2 DHT22 Sensor Firmware

The DHT22 sensor firmware is built around three main components: the Matter node and endpoint configuration in app_main(), the FreeRTOS polling task dht_task(), and the attribute update callbacks that push readings into the Matter stack.

In app_main(), the Matter node is created first, after which two separate endpoints are added - a temperature_sensor endpoint and a humidity_sensor endpoint. Each endpoint is created using the Esp-matter SDK's typed endpoint factory functions, which automatically add the correct clusters and attributes for that device type. The endpoint IDs assigned by the SDK are stored in g_temp_endpoint_id and g_humidity_endpoint_id as global variables, so the polling task can reference them when updating attribute values. The TFLM context classifier is initialised after the Matter node is configured but before esp_matter::start() is called, ensuring the model is ready before any sensor readings are taken [6].

Once the Matter stack is started, a FreeRTOS task called dht_task is created with a 4096 byte stack and priority 5. The task runs in an infinite loop, calling dht_read_float_data() every 2000 milliseconds to read temperature and humidity from the DHT22 on GPIO2. The 2 second minimum polling interval is a hardware constraint of the DHT22 - polling faster than this causes the sensor to return errors [8]. If the read succeeds, the temperature and humidity values are passed to their respective notification functions, and the TFLM classifier

is run on the same values. If the read fails, the error is logged and the task sleeps for the same interval before retrying.

The temp_sensor_notification() and humidity_sensor_notification() functions follow the same pattern. Rather than updating the Matter attribute directly from the FreeRTOS task context, they use chip::DeviceLayer::SystemLayer().ScheduleLambda() to post the update onto the Matter thread. This is a requirement of the CHIP SDK - attribute::update() is not thread-safe and must always be called from the Matter event loop thread, not from a FreeRTOS task. Inside the lambda, the existing attribute value is retrieved using attribute::get() and attribute::get_val(), the value is modified in place, and attribute::update() is called to push the new value into the Matter stack and trigger a subscription report to python-matter-server [6].

The Matter specification requires temperature values to be stored as signed 16-bit integers in units of 0.01°C, so a reading of 21.5°C is stored as 2150. Humidity values are stored as unsigned 16-bit integers in units of 0.01% RH, so 55.0% RH is stored as 5500. This conversion is applied in the notification functions by multiplying the float reading by 100 and casting to the appropriate integer type [4].

```
static void dht_task(void *pvParameters)
{
    float temperature = 0.0f;
    float humidity = 0.0f;

    while (true) {
        esp_err_t err = dht_read_float_data(DHT_TYPE, DHT_GPIO, &humidity, &temperature);

        if (err == ESP_OK) {
            temp_sensor_notification(g_temp_endpoint_id, temperature, nullptr);
            humidity_sensor_notification(g_humidity_endpoint_id, humidity, nullptr);

            ESP_LOGI(TAG, "DHT22: T=%.2fC H=%.2f%%", temperature, humidity);
```

Figure 7-1: dht_task reading & storing sensor values.

```
// Application cluster specification, 2.6.4.1. MeasuredValue Attribute
// represents the humidity in percent.
// humidity = (humidity in %) x 100
static void humidity_sensor_notification(uint16_t endpoint_id, float humidity, void *user_data)
{
    // schedule the attribute update so that we can report it from matter thread
    chip::DeviceLayer::SystemLayer().ScheduleLambda([endpoint_id, humidity]() {
        attribute_t * attribute = attribute::get(endpoint_id,
                                                 RelativeHumidityMeasurement::Id,
                                                 RelativeHumidityMeasurement::Attributes::MeasuredValue::Id);

        esp_matter_attr_val_t val = esp_matter_invalid(NULL);
        attribute::get_val(attribute, &val);
        val.val.u16 = static_cast<uint16_t>(humidity * 100);

        attribute::update(endpoint_id, RelativeHumidityMeasurement::Id, RelativeHumidityMeasurement::Attributes::MeasuredValue::Id,
    });
}
```

Figure 7-2: humidity notification callback function.

7.1.3 RGB LED Firmware

The LED node firmware is built around the extended_color_light device type from the Esp-matter SDK, which automatically provisions the OnOff, LevelControl, and ColorControl clusters on a single endpoint. In app_main(), the light endpoint is created with default values defined in app_priv.h - a default brightness of 64, default power state of true, and an initial colour mode of kColorTemperature. The endpoint ID is stored in the global light_endpoint_id variable, which app_driver.cpp uses to identify which endpoint's attribute updates should be forwarded to the LED hardware [1].

All hardware interaction is handled in app_driver.cpp through the app_driver_attribute_update() function, which is called from the app_attribute_update_cb() callback registered with the Matter stack. Every time a controller writes an attribute - whether that is the OnOff state, the brightness level, or the colour - the CHIP SDK calls this callback with the endpoint ID, cluster ID, attribute ID, and new value. The function checks which

cluster and attribute are being updated and dispatches to the appropriate driver function accordingly. app_driver_light_set_power() passes the boolean OnOff value directly to the LED driver. app_driver_light_set_brightness() remaps the Matter brightness range of 0-254 to the LED driver's range of 0-100 using the REMAP_TO_RANGE macro defined in app_priv.h before calling led_driver_set_brightness() [1].

The ColorControl cluster supports multiple colour modes - hue/saturation, colour temperature, and CIE XY. PiHub uses XY mode exclusively from the frontend, as described in Section 4.4.2. When the CurrentX or CurrentY attribute is updated, the new value is stored in the module-level globals current_x and current_y, and app_driver_light_set_xy() is called immediately with both values to update the hardware.

A significant bug was encountered during development where changing the LED brightness caused the colour to reset to white. The root cause was in the LevelControl attribute handler - when app_driver_light_set_brightness() was called, the LED driver internally reset its hardware colour state as a side effect of the brightness update, overwriting the current XY colour globals with default values. The fix was to call app_driver_light_set_xy(handle, current_x, current_y) immediately after app_driver_light_set_brightness() in the LevelControl handler, re-applying the stored XY coordinates after the brightness update.

```
} else if (cluster_id == LevelControl::Id) {
    if (attribute_id == LevelControl::Attributes::CurrentLevel::Id) {
        err = app_driver_light_set_brightness(handle, val);
        if (current_x > 0 || current_y > 0) {
            err |= app_driver_light_set_xy(handle, current_x, current_y);
        }
    }
```

Figure 7-3: Code snip of XY colour being set on a brightness change.

7.2 Backend Implementation

7.2.1 Implementation Challenges & Key Decisions

Several non-trivial problems were encountered during backend development, each requiring specific debugging and design decisions to resolve.

SQLite to PostgreSQL Migration

The backend was initially developed using SQLite as the database, chosen for its simplicity and zero-configuration setup. As the project matured, SQLite was replaced with PostgreSQL. The primary reason was concurrent write behaviour - with the Matter WebSocket listener writing sensor readings to the database on a background asyncio task at the same time as HTTP request handlers were performing reads and writes, SQLite's file-level locking caused occasional write failures under simultaneous access. PostgreSQL handles concurrent connections properly through its multi-process architecture and was a straightforward swap given the SQLAlchemy ORM abstraction layer already in place [11].

The Silent Coroutine Bug

One of the more difficult bugs to diagnose during development involved calling an async function without the await keyword. In Python, calling an async function without await does not execute it - it returns a coroutine object silently without raising any error or warning. This manifested as attribute update callbacks appearing to register successfully but never actually executing, since the function call that was meant to start the background listener was returning a coroutine object instead of running it. The fix was straightforward once

identified - adding the missing await keyword - but the silent failure mode made it difficult to locate [19].

start_listening Response Format

The python-matter-server WebSocket API returns the current state of all commissioned nodes in response to the start_listening command. The response schema returns the nodes as a list rather than a dict, which was not immediately obvious from the documentation. The initial implementation attempted to iterate the response as a dict, which caused the attribute cache to be populated incorrectly, resulting in live endpoint reads returning None for all values. The fix was to update the parsing logic in matter_ws.py to iterate the response as a list and index each node by its node_id field [18].

The run_in_executor Pattern

FastAPI's async event loop cannot be blocked by synchronous operations - doing so would stall all concurrent request handling. SQLAlchemy's synchronous Session cannot be used directly inside an async function without blocking the event loop. The solution was to wrap all database write operations in asyncio's run_in_executor(), which offloads the synchronous call to a threadpool and returns an awaitable, keeping the event loop free while the database operation completes. This pattern is used throughout main.py in the sensor persistence callbacks [10] [11].

HTTP Polling vs Frontend WebSocket

A deliberate decision was made to use HTTP polling from the React frontend rather than opening a WebSocket connection directly from the browser to the backend. For a kiosk application with a fixed 4-second refresh requirement, polling is simpler to implement, easier to reason about, and avoids the complexity of managing a persistent WebSocket connection in the frontend. The in-memory cache in matter_ws.py ensures that polls always return the most recent value without hitting the database.

7.3 Frontend Implementation

7.3.1 Colour Pipeline Implementation

Controlling the WS2812 LED colour from a hex colour picker in the browser required a non-trivial conversion pipeline. The Matter ColorControl cluster's XY colour mode represents colours using CIE 1931 chromaticity coordinates, which are a device-independent way of describing colour based on human perception. A hex colour string from the browser colour picker, such as #ff4400, is in the sRGB colour space, which is display-specific and gamma-encoded. Sending sRGB values directly to the Matter cluster would produce incorrect colours, so a full conversion pipeline was implemented in ColourHelpers.js.

The hexToXY() function first extracts the red, green, and blue channels from the hex string and normalises them to the range 0.0-1.0. It then applies sRGB gamma linearisation to each channel - values below 0.04045 are divided by 12.92, and values above that threshold use the standard sRGB piecewise function ((value + 0.055) / 1.055) ^ 2.4. This step converts the gamma-encoded sRGB values into linear light values, which is necessary before applying the colour matrix. The linearised RGB values are then multiplied by the Wide RGB D65 conversion matrix to produce CIE XYZ tristimulus values, from which the chromaticity coordinates x and y are derived by normalising X and Y against the sum X + Y + Z. The resulting x and y values are passed directly to the Matter MoveToColor command [4].

The reverse conversion, xyToHex(), was needed to display the current LED colour in the frontend colour picker when the dashboard first loads. This function takes the CIE XY values from the live cache and works backwards through the pipeline - reconstructing XYZ from XY, applying the inverse Wide RGB D65 matrix, clamping negative values, applying gamma encoding, and converting back to a hex string. A problem encountered with this round-trip was colour drift - the xyToHex conversion is lossy, and every time the frontend polled the live endpoint and updated the colour picker with the converted hex value, it would drift slightly from the original. If the user then changed brightness, the drifted colour would be sent back to the LED, compounding the error over time. The fix was the userSetColorRef flag in useLedState.js - once the user explicitly picks a colour, the flag is set and the polling loop stops updating the colour picker from the live cache, making the locally selected colour the source of truth for the duration of the session.

```
/**
 * Convert a hex colour string (#rrggbb) to CIE XY for the MoveToColor command.
 * Uses the sRGB -> XYZ -> xy conversion with the D65 white point.
 */
export function hexToXY(hex) {
  const r = parseInt(hex.slice(1, 3), 16) / 255;
  const g = parseInt(hex.slice(3, 5), 16) / 255;
  const b = parseInt(hex.slice(5, 7), 16) / 255;

  // Gamma correction (sRGB linearisation)
  const lr = r > 0.04045 ? Math.pow((r + 0.055) / 1.055, 2.4) : r / 12.92;
  const lg = g > 0.04045 ? Math.pow((g + 0.055) / 1.055, 2.4) : g / 12.92;
  const lb = b > 0.04045 ? Math.pow((b + 0.055) / 1.055, 2.4) : b / 12.92;

  // Wide RGB D65 conversion
  const X = lr * 0.664511 + lg * 0.154324 + lb * 0.162028;
  const Y = lr * 0.283881 + lg * 0.668433 + lb * 0.047685;
  const Z = lr * 0.000088 + lg * 0.072310 + lb * 0.986039;

  const sum = X + Y + Z;
  if (sum === 0) return { x: 0, y: 0 };
  return {
    x: parseFloat((X / sum).toFixed(6)),
    y: parseFloat((Y / sum).toFixed(6)),
  };
}
```

Figure 7-4: Code snip of HexToXY colour helper function.

7.3.2 Touch and Kiosk Optimisation

Running the React frontend in Chromium kiosk mode on a Raspberry Pi touchscreen introduced several implementation challenges that required specific solutions beyond standard web development practice.

The most pervasive change was replacing all :hover CSS pseudo-classes with :active throughout the stylesheet. On a touchscreen there is no cursor, so hover states never fire - interactive elements would appear completely unresponsive to touch without this change. The :active pseudo-class fires on physical contact with the screen, providing immediate visual feedback on press in the same way hover feedback works on desktop. This was applied globally across all buttons, tiles, nav links, and interactive elements in the stylesheet.

Chromium kiosk mode is launched automatically on Raspberry Pi startup via the labwc autostart file at ~/.config/labwc/autostart. The autostart entry launches Chromium with a set of command-line flags: --kiosk to enable full-screen kiosk mode with no browser chrome, --noerrdialogs and --disable-infobars to suppress any crash recovery or update prompts that would otherwise appear over the UI, and --app pointing to the local Nginx address where the frontend is served. Without the --noerrdialogs flag, a Chromium crash recovery dialog can appear on startup after an unclean shutdown and block the entire UI until manually dismissed, which is not acceptable for a kiosk device.

A bug was found and fixed in the commissioning flow in Settings.jsx. The original implementation fired a success toast notification after the commission API call completed, regardless of whether the call had actually succeeded. This happened because the response was not being checked for a valid node_id before triggering the notification - if the API returned an error or an unexpected response shape, the success message would still appear, giving the user false confidence that the device had been commissioned when it had not. The fix was to check that the response contained a valid node_id before showing the success message, and to show a specific error message in all other cases.

7.4 Containerisation and Deployment

The two-stack Docker Compose architecture and CI/CD pipeline are described in detail in Sections 4.7 and 5.3.5. A few specific implementation issues are worth noting here.

One early issue with the PostgreSQL configuration was that an @ symbol in the database password caused URL parsing to fail silently in the SQLAlchemy connection string. The @ character is a reserved delimiter in database URLs, so it was interpreted as the separator between the credentials and the host, producing a malformed connection string that was difficult to diagnose. The fix was simply to use a password without special characters.

A related operational note is that running docker compose down -v - the flag that removes named volumes along with containers - permanently destroys the PostgreSQL data volume, including all persisted device name registrations. This is not obvious from the command itself and caused data loss during early deployment iterations. For normal restarts and updates, docker compose down without the -v flag should always be used, with --pull handling image updates instead.

The CI/CD pipeline builds multi-platform Docker images targeting both linux/amd64 and linux/arm64 architectures in a single workflow run using Docker Buildx. This is necessary because the development machine runs amd64 and the Raspberry Pi runs arm64 - without multi-platform builds, images pushed from the laptop would not run on the Pi. Layer caching is configured using GHCR as the cache registry, which significantly reduces build times on subsequent pushes by reusing unchanged layers [12] [13].

7.5 TinyML On-Device Context Classification

7.5.1 Problem Definition and Class Design

The TinyML workstream was motivated by the opportunity to demonstrate that a microcontroller-class device can do more than just report raw sensor values - it can interpret them locally and surface a meaningful classification result without any cloud involvement or round-trip to the hub.

The classification problem is defined as follows: given a pair of real-time DHT22 readings - temperature in degrees Celsius and relative humidity as a percentage - classify the current environmental context into one of three discrete classes. HEATING_ON represents conditions where the heating system is active, characterised by elevated temperature and stable or slowly falling humidity as the air dries out. NORMAL represents typical ambient indoor conditions with no significant environmental event occurring. WINDOW_OPEN represents conditions where a window has been opened, characterised by a drop in temperature and a rise in humidity as outdoor air enters the room.

These three classes were chosen because they represent the most common and meaningfully distinct environmental states in an Irish domestic setting, and because their boundaries can be described in terms of known physical conditions rather than statistical

anomalies. This made them well suited to a supervised classification approach, where the training data could be generated synthetically from physical reasoning about each class rather than requiring weeks of labelled real-world data collection.

The result of the inference is exposed through a Matter attribute on the sensor node, as described in Section 7.5.6, meaning the classification happens entirely on the ESP32-C3 and is reported to the hub in the same way as any other sensor reading. There is no cloud round-trip, no hub-side inference, and no dependency on the Raspberry Pi being reachable for the classification to run.

7.5.2 Data Collection and Synthetic Augmentation

The initial data collection approach used a pyserial script reading directly from /dev/ttyUSB0, capturing real DHT22 readings from the ESP32-C3 and exporting them as a CSV file. This gave a dataset of real sensor readings from the physical environment, which was used for a first round of training.

The problem with real-world data became apparent when the class separation plot from the first training run was examined, shown in Figure 7-5. The three classes are extremely tightly clustered with almost no spread - NORMAL sits in a tiny blob around 18°C/61%, WINDOW_OPEN clusters tightly around 13°C/79%, and HEATING_ON clusters tightly around 22-24°C/45%. This is exactly what would be expected from a short data collection session where the ambient conditions barely changed. The temperature axis spans only 13°C to 24°C, and because the data was collected over a limited period without deliberately inducing each context, the model had no training examples near the class boundaries. Any reading that fell between these tight clusters would be misclassified with high confidence.

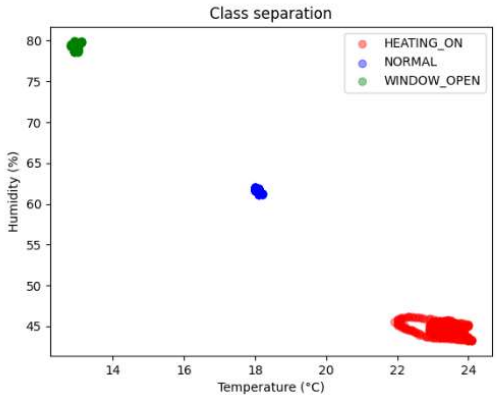


Figure 7-5: Plot showing real-world data collection, showing tight clustering and poor temperature coverage.

The solution was to abandon the real-world dataset and retrain entirely on synthetic Gaussian data generated from physical reasoning about each class. For HEATING_ON, a higher mean temperature and lower humidity was used to reflect warm, dry indoor air when the heating is active. For WINDOW_OPEN, a lower mean temperature and higher humidity was used to reflect cool, moist outdoor air entering the room. For NORMAL, a mid-range temperature and moderate humidity was used to reflect typical ambient indoor conditions. A standard deviation was applied to each feature to generate a realistic spread of values around each class centre, with the parameters chosen to ensure sufficient overlap at the boundaries without the model becoming overconfident near the decision boundary.

The result is shown in Figure 7-6. Compared to Figure 7-5, the classes now occupy well-separated but realistically spread regions across a much wider temperature range of 12°C to

30°C and humidity range of 40% to 85%. Each class forms a natural Gaussian cloud rather than a tight pinpoint cluster, giving the model the coverage it needs to generalise to real sensor readings across the full range of conditions it will encounter in deployment. Classification accuracy improved significantly after retraining on this dataset.

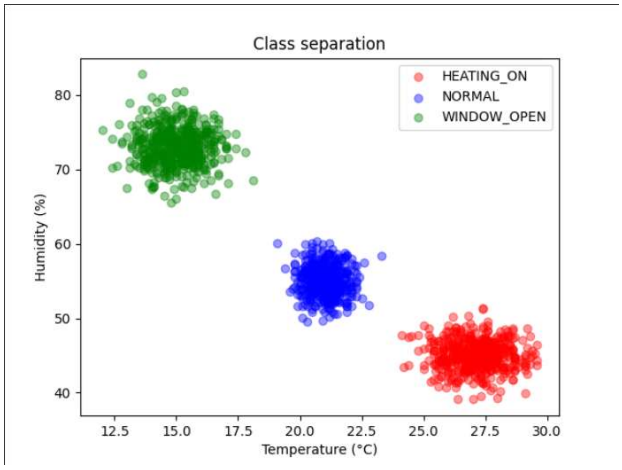


Figure 7-6: Plot showing dummy data collection, showing realistic spread and clear separation.

7.5.3 Model Architecture and Training

The classifier is a shallow feed-forward neural network (MLP) defined using Keras. The architecture consists of an input layer accepting two features (temperature and humidity), a single hidden layer of 8 neurons with ReLU activation, and an output layer of 3 neurons with softmax activation corresponding to the three classes. The full model summary from the training run is shown below - the network has just 51 trainable parameters in total, making it extremely compact and well suited to int8 quantisation for TFLM deployment [14].

Before training, the input features are normalised using scikit-learn's MinMaxScaler, which scales each feature to the range 0.0-1.0 based on the minimum and maximum values seen in the training data. The scaler is fitted on the full dataset before the train/test split, and the resulting constants are printed and embedded in the firmware. This is a critical step - the ESP32-C3 has no Python runtime, so the same normalisation transform must be re-implemented in C using fixed-point constants rather than calling the scaler at runtime. The constants produced by the training run were:

```
Scaler min_:   [-0.68181818, -0.89473684]
Scaler scale_: [ 0.05681818,  0.02288330]
```

Figure 7-7: Normalised input features.

These correspond to the temperature and humidity features respectively and are embedded in ml_context.cpp as constexpr float values used to normalise each sensor reading before it is passed to the TFLM interpreter.

The synthetic Gaussian training data was generated with the following class distributions, derived from physical reasoning about each context:

- **NORMAL** - temperature mean 21.0°C, scale 0.6; humidity mean 55.0%, scale 2.0. Represents typical Irish indoor ambient conditions.
- **HEATING_ON** - temperature mean 27.0°C, scale 1.0; humidity mean 40.0%, scale 2.0. Represents elevated temperature and drier air when a radiator is active nearby.

- **WINDOW_OPEN** - temperature mean 15.0°C, scale 1.0; humidity mean 70.0%, scale 2.5. Represents cool, damp outdoor air entering the room.

500 samples were generated per class, giving a balanced dataset of 1500 total samples. The dataset was split 80/20 into training and test sets using stratified sampling to ensure each split had proportional class representation. The model was compiled with the Adam optimiser and sparse categorical cross-entropy loss and trained for 100 epochs with a batch size of 16.

Training converged quickly - by epoch 10 validation accuracy had reached 100%, and it remained at 100% for all remaining epochs. Final test accuracy was 100.00%, which is expected given the clean Gaussian separation between classes shown in Figure 7-6. The model was not overfitting in a meaningful sense - the class boundaries are genuinely well-separated in the feature space, and the model is simply learning a clean linear decision boundary between them.

7.5.4 Model Quantisation and Firmware Export

Once training was complete, the Keras model was converted to a TensorFlow Lite model with int8 post-training quantisation using the TFLiteConverter API. The converter was configured with DEFAULT optimisations, which enables quantisation, and a representative dataset function was provided so the converter could calibrate the quantisation parameters by observing the range of activations across the full training set. The target spec was set to TFLITE_BUILTINS_INT8, and both the input and output types were set to tf.int8, producing a fully quantised model where all weights, activations, inputs, and outputs are represented as 8-bit integers [14].

One additional flag was required during conversion - _experimental_disable_per_channel_quantization_for_dense_layers set to True. Without this, the TFLM AllOpsResolver on the ESP32-C3 could not execute the quantised dense layers correctly due to per-channel quantisation not being supported for dense operations in the version of TFLM used. This is a known compatibility issue between certain TensorFlow versions and older TFLM builds.

The resulting quantised model was 2128 bytes - a very small binary that fits comfortably within the ESP32-C3's 4 MB flash. For reference, the unquantised float32 model would have been approximately four times larger at around 8 KB, though both are trivially small compared to the available flash. The size advantage of quantisation matters more for the RAM footprint during inference, where the int8 model requires a significantly smaller tensor arena.

The .tflite binary was converted to a C header file using the xxd command-line tool. xxd produces a hex dump of a binary file, and the -i flag formats the output as a valid C array and unsigned integer length declaration rather than a human-readable hex dump. The command used was “xxd -i context_dummy_model.tflite > model_data.h”, which produces a header file containing an unsigned char array of the raw model bytes and an unsigned int holding the byte count - in this case 2128. This header is included directly in the firmware source, and the byte array is passed to the TFLM MicroInterpreter at runtime. No filesystem or SD card access is required - the entire model is baked into the firmware binary at compile time and stored in flash alongside the application code.

```python
# Export to TFLite int8
def representative_dataset():
    for sample in X_scaled:
        yield [sample.reshape(1, 2).astype(np.float32)]

converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.representative_dataset = representative_dataset
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.int8
converter.inference_output_type = tf.int8

converter._experimental_disable_per_channel_quantization_for_dense_layers = True

tflite_model = converter.convert()

with open("context_dummy_model.tflite", "wb") as f:
    f.write(tflite_model)

print(f"Model size: {len(tflite_model)} bytes")
```

Figure 7-8: Quantisation section in training script.

7.5.5 TFLM Firmware Integration

The TFLM inference pipeline is implemented in ml_context.cpp, split into two functions: ml_context_init() which sets up the interpreter once at startup, and ml_context_run() which is called on every sensor reading to produce a classification.

ml_context_init() begins by loading the model from the context_model_tflite byte array defined in context_model_data.h - the file generated by the xxd command described in Section 7.5.4. Rather than using the AllOpsResolver, which links the entire TFLM operator library into the firmware, a MicroMutableOpResolver is used with only the three operations the model actually needs: FullyConnected for the dense layers, Relu for the hidden layer activation, and Softmax for the output layer. This keeps the firmware flash footprint smaller by avoiding linking unused operator code. A static tensor arena of 4 KB is allocated as a fixed uint8 array - this is the block of RAM the interpreter uses for all intermediate tensor allocations during inference. The arena size was determined empirically; the init log reports actual usage, which for a 51-parameter model is well under 1 KB [14].

ml_context_run() takes a raw temperature and humidity float pair from the DHT22 and runs inference in four steps. First, the MinMaxScaler normalisation is applied to each input using the constants embedded from the training output - the formula is X_scaled = X * scale_ + min_, which must exactly match the sklearn MinMaxScaler transform applied during training. Second, the normalised float values are quantised to int8 using the scale and zero_point parameters embedded in the input tensor metadata by the TFLite converter, following the formula q = float / scale + zero_point. Third, s_interpreter->Invoke() runs the forward pass through the network entirely within the tensor arena. Fourth, the output tensor is read - each of the three int8 values is dequantised back to a float probability using the output tensor's scale and zero_point, and the index with the highest probability is returned as the predicted class. The raw int8 comparison would also be valid for finding the argmax, but dequantising first makes the confidence value available for the ESP_LOGI log output, which was useful during development for confirming the model was producing sensible probabilities [14].

The three class labels are mapped by index in the kLabels array - 0 = HEATING_ON, 1 = NORMAL, 2 = WINDOW_OPEN - matching the alphabetical order produced by sklearn's LabelEncoder during training. The predicted class index is returned to dht_task() in app_main.cpp, where it is used to update the MinMeasuredValue attribute workaround described in Section 7.5.6.

```
int ml_context_run(float temperature, float humidity)
{
    if (!s_interpreter) {
        ESP_LOGE(TAG, "Interpreter not initialised — call ml_context_init() first");
        return -1;
    }

    // — 1. Apply MinMaxScaler normalisation ————————————————————————
    // Must exactly match sklearn's MinMaxScaler: X_scaled = X * scale_ + min_
    float t_scaled = temperature * SCALER_SCALE_TEMP + SCALER_MIN_TEMP;
    float h_scaled = humidity * SCALER_SCALE_HUM  + SCALER_MIN_HUM;

    // — 2. Quantise float inputs to int8 ————————————————————————————
    // TFLite quantisation formula: q = float / scale + zero_point
    // scale and zero_point are embedded in the model by the TFLite converter
    // and read directly from the input tensor metadata.
    float in_scale = s_input->params.scale;
    int32_t in_zero_point = s_input->params.zero_point;
    s_input->data.int8[0] = static_cast<int8_t>(t_scaled / in_scale + in_zero_point);
    s_input->data.int8[1] = static_cast<int8_t>(h_scaled / in_scale + in_zero_point);

    // — 3. Run inference ————————————————————————————————————————————
    if (s_interpreter->Invoke() != kTfLiteOk) {
        ESP_LOGE(TAG, "Invoke() failed");
        return -1;
    }

    // — 4. Find predicted class ————————————————————————————————————
    // The output tensor has one int8 value per class after softmax.
    // The class with the highest value has the highest probability.
    // We compare raw int8 values directly — no dequantisation needed
    // since we only care about which index is largest, not the actual probability.
    float out_scale = s_output->params.scale;
    int32_t out_zero_point = s_output->params.zero_point;

    float max_prob = (s_output->data.int8[0] - out_zero_point) * out_scale;
    int predicted  = 0;

    for (int i = 1; i < kNumClasses; i++) {
        float prob = (s_output->data.int8[i] - out_zero_point) * out_scale;
        if (prob > max_prob) {
            max_prob  = prob;
            predicted = i;
```

Figure 7-9: ml_context_run() function.

7.5.6 Custom Matter Vendor Cluster - Context Attribute

The intended approach for exposing the TinyML classification result to python-matter-server was to define a custom vendor-specific cluster on the temperature sensor endpoint. The Matter specification reserves cluster IDs in the range 0xFC00 to 0xFFFE for manufacturer use, and a cluster with ID 0xFC00 was created in app_main.cpp with a single float attribute at attribute ID 0x0000 to hold the predicted class index. The cluster was given the mandatory global attributes - ClusterRevision (0xFFFD) and FeatureMap (0xFFFC) - which are required by the CHIP SDK for a cluster to be considered well-formed. Without these, the SDK rejects the cluster as malformed and drops all its attribute reports silently [4].

However, even with a correctly formed vendor cluster, attribute reports were not reaching python-matter-server. After extensive debugging - monitoring the raw WebSocket messages from python-matter-server and confirming that the firmware was updating the attribute correctly via serial logs - it was established that python-matter-server 8.1.0 was silently discarding attribute update events for vendor cluster IDs. This was traced to a confirmed bug in the connectedhomeip SDK, tracked as GitHub issue #32371, whereby the attribute reporting engine rejects vendor cluster attribute paths during subscription processing. The vendor cluster attribute updates were being generated correctly by the firmware but never forwarded to the backend [18].

A workaround was required that would carry the classification result over a standard, well-supported Matter attribute that python-matter-server would not discard. The MinMeasuredValue attribute (0x0001) on the RelativeHumidityMeasurement cluster at endpoint 2 was chosen for this purpose. This attribute is part of the standard humidity cluster and is subscribed to normally by python-matter-server, but it is not used for any meaningful humidity measurement in PiHub - its intended purpose is to report the minimum measurable humidity value of the sensor, which is a static hardware characteristic rather than a live reading. By repurposing it to carry the integer class index instead, the classification result can be transmitted through the standard subscription mechanism without any changes to the python-matter-server subscription configuration.

On the firmware side, after each inference the class index is written to the MinMeasuredValue attribute using the standard attribute::get() and attribute::update() pattern, as shown in the dht_task() function in app_main.cpp. On the backend side, matter_ws.py registers a callback for the attribute path "2/1029/1" - endpoint 2, RelativeHumidityMeasurement cluster (0x0405), attribute 1 - which fires whenever a new classification result arrives. The callback stores the class index in the sensor reading database as sensor_type "context" and also updates the in-memory live cache. The FastAPI /live endpoint returns the context class index alongside temperature and humidity, and the React frontend maps it to a human-readable label and badge icon on the SensorTile component using the CONTEXT_STYLE lookup table.

```
// Run TFLM context classifier and update the custom attribute.
// ScheduleLambda posts the attribute update onto the Matter thread —
// attribute::update must not be called from a FreeRTOS task directly.
int context = ml_context_run(temperature, humidity);
if (context >= 0) {
    chip::DeviceLayer::SystemLayer().ScheduleLambda([context]() {
        // Primary: update vendor cluster (0xFC00) — correct approach per Matter spec.
        // Currently skipped by python-matter-server 8.1.0 due to a known CHIP SDK
        // bug (connectedhomeip issue #32371) which rejects vendor cluster IDs.
        esp_matter_attr_val_t val = esp_matter_float(static_cast<float>(context));
        attribute::update(g_context_endpoint_id,
                          kContextClusterId,
                          kContextAttributeId,
                          &val);

        // Workaround: write context class to MinMeasuredValue (0x0001) on the
        // humidity endpoint using the same get/modify/update pattern as
        // MeasuredValue — this ensures the subscription engine reports it.
        attribute_t *min_attr = attribute::get(
            g_humidity_endpoint_id,
            RelativeHumidityMeasurement::Id,
            RelativeHumidityMeasurement::Attributes::MinMeasuredValue::Id
        );
        if (min_attr) {
            esp_matter_attr_val_t workaround_val = esp_matter_invalid(NULL);
            attribute::get_val(min_attr, &workaround_val);
            workaround_val.val.u16 = static_cast<uint16_t>(context);
            attribute::update(g_humidity_endpoint_id,
                              RelativeHumidityMeasurement::Id,
                              RelativeHumidityMeasurement::Attributes::MinMeasuredValue::Id,
                              &workaround_val);
```

Figure 7-10: context class label attribute assignment.

8. Testing and Evaluation

8.1 Testing Strategy

Testing in PiHub was approached across different layers, each requiring a different methodology due to the nature of the components involved.

Frontend testing was automated using Vitest, the Vite-native testing framework. Vitest was chosen over Jest because it shares the same configuration as the Vite build tool - no separate Jest config, no babel transform setup, and no import resolution conflicts with ES modules [20]. The test suite focuses on pure helper functions and stateless components, particularly the colour conversion pipeline in ColourHelpers.js which contains non-trivial mathematical logic that benefits from unit testing.

System-level testing was carried out end-to-end with the full stack running on the Raspberry Pi - both nodes commissioned, Docker Compose stacks running, and the React dashboard loaded in Chromium kiosk mode. This validated the complete data flow from sensor reading through to dashboard display and confirmed that LED control commands reached the hardware correctly.

Both the pytest and Vitest suites are integrated into the GitHub Actions CI/CD pipeline as parallel jobs that run on every push to main. The Docker build and push job will not execute unless both test jobs pass, meaning a failing test prevents a broken image from being published to GHCR. This is discussed in detail in Section 4.7.2 and shown in Figure 4-7 [13].

8.2 Backend Testing - pytest

The pytest test suite is organised into seven test modules under the tests/ directory, each targeting a distinct area of the backend. The conftest.py setup is minimal - it defines a single client fixture that returns a FastAPI TestClient instance wrapping the application. Tests that require a real database override FastAPI's get_db dependency with an in-memory SQLite session, replacing the PostgreSQL connection entirely so no database container is needed during CI. Tests that involve Matter WebSocket calls mock the relevant matter_ws functions using unittest.mock's patch and AsyncMock, so no live Matter network or python-matter-server instance is required [10].

The test modules and what they cover are as follows. test_health.py verifies the /health endpoint returns a 200 response with the correct JSON body. test_devices.py covers the full device registry - listing devices, registering new devices, duplicate node_id enforcement (409), input validation on name length (422), deletion, and isolation between node IDs. test_sensors.py covers the sensor history endpoint - empty results, filtering by sensor_type, the limit parameter, newest-first ordering, node ID isolation, and correct response field shapes including context readings. test_live_endpoints.py covers all live cache endpoints and LED control commands, with all matter_ws functions mocked. test_matter_ws_cache.py tests the pure Python cache helper functions in matter_ws.py directly, including _cache_node, _evict_node, *find_cached, and all the get_cached** functions. test_persist_and_callbacks.py tests the _persist_sensor_reading function and the three sensor attribute callbacks for temperature, humidity, and context. test_ws_commands.py tests all Matter command wrapper functions by patching _ws_call with AsyncMock.

A representative test from test_persist_and_callbacks.py illustrates the arrange-act-assert pattern used throughout the suite. The test below verifies that the temperature callback correctly converts a raw Matter value to degrees Celsius and persists it to the database:

The test first calls _register_sensor_callbacks([1]), which registers the on_temperature callback for attribute path "1/1026/0" on node 1. It then retrieves the registered callback from

the _subscribers dict and runs it using asyncio.run(), passing in a raw Matter temperature value of 2150. This represents 21.5°C - the Matter specification encodes temperature as hundredths of a degree, so 2150 / 100 = 21.5. The test then opens a fresh database session and queries for the persisted row, asserting that the sensor_type is "temperature_c" and the value is approximately 21.5. The pytest.approx() matcher is used for the float comparison to avoid floating point precision issues.

```python
def test_temperature_callback_converts_raw_and_persists(self):
    with patch("app.main.SessionLocal", TestingSessionLocal):
        _register_sensor_callbacks([1])
        cb = _subscribers[(1, "1/1026/0")][0]
        asyncio.run(cb(1, "1/1026/0", 2150))

    db = TestingSessionLocal()
    row = db.query(SensorReadingDB).filter_by(sensor_type="temperature_c").first()
    db.close()

    assert row is not None
    assert row.value == pytest.approx(21.5)
```

Figure 8-1: test_temperature_callback_converts_raw_and_persists() test function.

```
Name                Stmts   Miss  Cover   Missing
-----------------------------------------------
app/__init__.py         0      0   100%
app/database.py        29      4    86%   54-60
app/main.py           143     12    92%   46-50, 71-73, 138-146
app/matter_ws.py      178     74    58%   107-205, 234, 284-300
app/models.py          17      0   100%
app/schemas.py         33      0   100%
-----------------------------------------------
TOTAL                 400     90    78%
```

Figure 8-2: pytest coverage report.

The key test cases across the suite are summarised in the table below.

Test Name	Module	Endpoint/Function	Description
test_register_device_returns_201	test_devices	POST /api/devices	Verifies device registration returns 201
test_register_duplicate_node_id_returns_409	test_devices	POST /api/devices	Verifies duplicate node_id returns 409
test_register_device_name_too_long_returns_422	test_devices	POST /api/devices	Verifies name over 50 chars returns 422
test_unregister_nonexistent_device_returns_404	test_devices	DELETE /api/devices/{id}	Verifies missing device returns 404
test_sensor_history_filter_by_sensor_type	test_sensors	GET /api/sensors/{id}/history	Verifies sensor_type filter returns only matching rows
test_sensor_history_ordered_newest_first	test_sensors	GET /api/sensors/{id}/history	Verifies results are ordered newest first
test_sensor_history_respects_limit	test_sensors	GET /api/sensors/{id}/history	Verifies limit parameter is applied

test_live_sensors_no_cache_returns_404	test_live_endpoints	GET /live sensors	Verifies 404 when cache is empty
test_set_brightness_clamps_overflow_to_254	test_ws_commands	set_brightness()	Verifies brightness is clamped to 254 max
test_context_callback_persists_class_id	test_persist	on_context callback	Verifies TinyML class index is persisted as context reading
test_humidity_callback_ignores_none	test_persist	on_humidity callback	Verifies None values are not written to the database

8.3 Frontend Testing - Vitest

The Vitest test suite is configured in vite.config.js under the test key. The test environment is set to jsdom, which simulates a browser DOM environment so that React components and browser APIs can be tested without a real browser. The globals flag is set to true, meaning describe, it, expect, and vi are available in every test file without explicit imports. A setup file at src/test/setup.js imports @testing-library/jest-dom, which extends Vitest's expect with DOM-specific matchers such as toBeInTheDocument(). Coverage is collected using the v8 provider across all .js and .jsx files under src/, excluding the test directory and main.jsx entry point.

The test suite is organised into seven test files. ColourHelpers.test.js and ChartHelpers.test.jsx test pure helper functions and the ChartTooltip component. client.test.js tests the shared fetch wrapper. devices.test.js, matter.test.js, and sensors.test.js test the three API modules by mocking the client module with vi.mock(). useLedState.test.js tests the LED state hook using @testing-library/react's renderHook and act utilities, with all Matter API functions mocked.

The primary targets for testing were pure functions and stateless logic rather than fully rendered page components. This was made possible by the refactoring work described in Section 7.3.1 - before ColourHelpers.js was extracted from the component tree, the hexToXY and xyToHex functions were embedded inside a component and could not be imported and tested in isolation. Extracting them into a standalone module was a prerequisite for unit testing this logic.

The ColourHelpers.test.js suite is a strong illustration of the testing approach. The hexToXY() tests verify the colour conversion pipeline against known CIE XY values for specific inputs. For example, the test for black (#000000) asserts that hexToXY returns {x: 0, y: 0}, since a zero-luminance input produces a zero sum in the XYZ conversion and the function explicitly handles this edge case by returning early. The white test verifies that hexToXY('#ffffff') returns x approximately 0.323 and y approximately 0.329, which are the CIE 1931 XY coordinates of the D65 white point. The blue test verifies that the x and y values are both below 0.2 and 0.1 respectively, reflecting blue's position in the bottom-left of the CIE chromaticity diagram. A loop test verifies that x and y values are always in the range 0.0 to 1.0 across red, green, blue, white, and mid-grey inputs. The xyToHex() tests verify the round-trip behaviour - converting a hex colour to XY and back should produce a result where the dominant channel is preserved. For red, the round-trip result should have a higher red channel value than green or blue.

The useLedState.test.js suite tests the hook's debouncing behaviour, optimistic state updates, and the userSetColorRef protection. The debounce tests use Vitest's fake timers - vi.useFakeTimers() freezes the timer system, and vi.advanceTimersByTime(299) confirms that the API is not called before the 300ms debounce window expires, while advancing by 300ms confirms it is called exactly once with the final value after rapid successive calls. The

userSetColorRef protection test verifies that once the user sets a colour via changeColor(), the subsequent poll cycle does not overwrite it with the converted xyToHex value from the live cache.

File	% Stmts	% Branch	% Funcs	% Lines
src/api/	100	100	100	100
ColourHelpers.js	100	100	100	100
ChartHelpers.jsx	91.3	100	75	88.88
useLedState.js	88.13	94.44	78.57	92.15
src/pages/*.jsx	0	0	0	0
src/components/*.jsx	0	0	0	0
ThemeContext.jsx	0	0	0	0
TOTAL	35.91	35.76	30	34.98

Figure 8-3: Vitest coverage report.

8.4 System Integration Testing

System integration testing was carried out manually with the full PiHub stack running on the Raspberry Pi - both Docker Compose stacks active, both ESP32-C3 nodes commissioned and online, and the React dashboard loaded in Chromium kiosk mode. Rather than scripted test cases, integration testing took the form of end-to-end validation of each data path described in Section 5.4.

For the sensor ingestion path, validation involved confirming that a DHT22 reading on the ESP32-C3 resulted in a visible update on the dashboard within the 4-second polling interval, and that the same reading appeared in the sensor history chart after being queried from PostgreSQL. The TinyML context classification path was validated by manually inducing each of the three context conditions - running a heat source near the sensor for HEATING_ON, opening a window for WINDOW_OPEN, and observing idle conditions for NORMAL - and confirming that the correct context badge appeared on the SensorTile.

For the LED control path, validation involved sending colour, brightness, and toggle commands from the dashboard and confirming that the WS2812 LED responded correctly and that the live state endpoint reflected the updated values on the next poll.

Several integration failures were encountered and resolved during this process. The start_listening response parsing issue, where the node list was incorrectly treated as a dict, only became apparent during full-stack testing when live endpoint reads returned None for all cached values despite the nodes being online. This is described in detail in Section 7.2.1. The commissioning silent failure in Settings.jsx, where a success toast fired regardless of the API response, was also identified during integration testing when a failed commission attempt appeared to succeed in the UI. This is described in Section 7.3.2.

8.5 UI and Kiosk Testing

User acceptance testing of the React dashboard was carried out directly on the Raspberry Pi touchscreen, validating touch responsiveness, readability, kiosk mode behaviour, and autostart reliability.

Touch responsiveness was validated across all interactive elements - tile taps, the brightness slider, the colour picker, the toggle switch, and the navigation bar. The replacement of :hover with :active pseudo-classes, described in Section 7.3.2, was confirmed to provide consistent visual feedback on touch press across all elements. The brightness slider and colour picker both required specific attention as they involve

continuous touch-drag gestures rather than discrete taps - both were confirmed to respond correctly to drag input on the touchscreen without requiring any additional touch event handling beyond the standard HTML input elements.

Readability at arm's length was assessed with the dashboard mounted in a fixed position as it would be in normal use. Font sizes and tile dimensions were adjusted iteratively during development until all key values - temperature, humidity, LED state, and context badge - were legible from a normal viewing distance without requiring the user to lean in.

Several issues were encountered when configuring Chromium kiosk mode. True kiosk mode, launched with the --kiosk flag, hides the browser chrome entirely and disables the on-screen keyboard that Chromium would normally provide for text input fields. This made the commissioning form in Settings.jsx unusable in true kiosk mode, since there was no way to type a pairing code without a physical keyboard attached. The solution was to use full-screen mode instead of true kiosk mode - launching Chromium with --start-fullscreen rather than --kiosk, which hides the top bar and address bar but preserves the on-screen keyboard for text input when a form field is focused.

A second issue encountered was that touch scrolling was not working at all on the dashboard. Tapping buttons worked correctly, but dragging a finger to scroll the page had no effect, and the side scrollbar could not be dragged with a finger either. After investigating on the Raspberry Pi forums, the cause was identified as the touchscreen being set to mouse emulation mode, which maps touch input to mouse events rather than native touch events. Changing the touchscreen mode from mouse emulation to multitouch resolved the issue and allowed the page to be scrolled normally with a finger [21].

Chromium autostart was configured via the labwc autostart file at ~/.config/labwc/autostart, which launches Chromium after the pihub.sh startup script confirms the frontend is ready on port 80. Autostart stability was tested across multiple cold boots and unclean shutdowns - the --noerrdialogs and --disable-infobars flags were confirmed to suppress the crash recovery dialog that Chromium displays after an unclean shutdown, which would otherwise block the UI on the next boot.

9. Hardware Enclosures

9.1 Requirements and Selection Criteria

Three enclosures were required for the PiHub hardware: two identical cases for the ESP32-C3-DevKitM-1 nodes, and a combined housing and stand for the Raspberry Pi 4 and touchscreen display.

For the ESP32-C3 nodes, the key requirements were compatibility with the DevKitM-1 board dimensions, clearance for the micro-USB port for power and flashing and an unobstructed window over the onboard WS2812 LED on the LED node. The sensor node had no additional hardware requirements since the DFRobot DHT22 module is connected externally via wires rather than enclosed with the board. Designing a custom enclosure from scratch was considered but ruled out due to time constraints and the availability of well-validated community designs for common ESP32 form factors. The ESP32-C3 case by Printables user drandreww was selected, which is designed specifically for the DevKitM-1 board and meets all of the above requirements [22]. Two identical prints were produced for both nodes.

For the Raspberry Pi, the requirements were a stable stand that would hold the touchscreen at a comfortable viewing angle, access to the USB-C power port and USB ports, and a clean housing that kept the board protected during use on a desk. A combined case and stand designed for the Raspberry Pi Touch Display 2 and Raspberry Pi 4 was selected from Printables user Iain White [23], providing a purpose-built housing that holds the screen at a fixed angle and routes the Pi neatly behind it.

9.2 3D Printing

All enclosures were printed by Daniel Kell. The ESP32-C3 enclosures were straightforward - both cases were printed in a single run without any issues and fit the DevKitM-1 boards correctly on the first attempt.

The Raspberry Pi enclosure required a couple of iterations. The initial print used an incorrect model - a case designed for the original Raspberry Pi 7" Touchscreen Display (Touch Display 1) rather than the newer Touch Display 2, which has different dimensions [24]. This resulted in a housing where the screen did not fit correctly.

Figure 9-1: Incorrect initial print.

Once the correct model was identified and the updated STL files were sent for printing, the first print of the correct model failed part-way through, but it was sufficient to do a test fit and

confirm the dimensions were right. The final reprint completed successfully, and the Pi, screen, and board all fit correctly within the housing.

Figure 9-2: Correct model failed first print.

9.3 Final Assembly

The completed enclosures with the nodes installed are shown in Figure 9-3. Both ESP32-C3-DevKitM-1 boards fit correctly within their cases, with the micro-USB port accessible for power and flashing and the onboard WS2812 LED visible through the LED window on the actuator node case. The Raspberry Pi 4 and Touch Display 2 are housed in the combined stand, which holds the screen at a comfortable viewing angle and keeps the Pi secured behind the display.

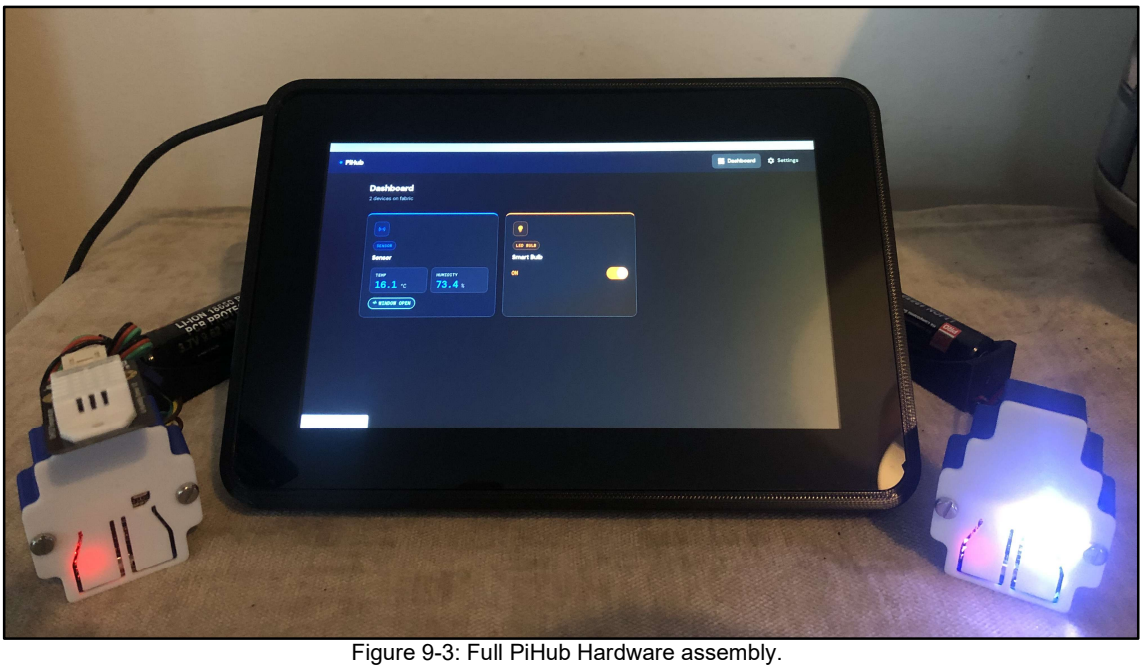


Figure 9-3: Full PiHub Hardware assembly.

10. Ethics

10.1 Data Privacy

PiHub collects environmental sensor data - temperature, humidity, and a derived context classification - from within a domestic home environment. While this data is not personally identifiable in the traditional sense, it does capture information about activity patterns and conditions within a private residence. Regular temperature and humidity fluctuations can implicitly reveal occupancy patterns, heating habits, and daily routines, which could be considered personal data under GDPR if they could be linked to an identifiable individual [25].

The key privacy distinction of PiHub is that all data remains entirely local. Sensor readings are stored in a PostgreSQL database running on the Raspberry Pi itself, with no cloud transmission, no third-party analytics, and no external API calls at runtime. This is a fundamental architectural difference from most commercial smart home systems, where sensor data is routinely transmitted to and stored on manufacturer-hosted servers. In PiHub, the user has full ownership and control of their data at all times.

From a GDPR perspective, since PiHub is a self-hosted personal project operated by the same individual whose home is being monitored, the household exemption under Article 2(2)(c) of the GDPR applies, meaning the regulation does not impose formal compliance obligations in this context. However, if PiHub were to be deployed in a shared or commercial environment, a data retention policy would be appropriate - defining how long sensor readings are stored in the database and providing a mechanism for deletion.

10.2 Security and Attack Surface

The security architecture of PiHub at the Matter protocol and Docker network layers is discussed in detail in Section 5.5. This section focuses on the broader security considerations and potential attack vectors.

The primary attack surface is the local network. The FastAPI backend is exposed on port 8000 and the Nginx frontend on port 80, both accessible to any device on the same Wi-Fi subnet. There is currently no authentication on the API, meaning any device on the local network could send LED control commands or trigger a commissioning attempt. For a single-user home deployment on a trusted network this is an acceptable trade-off, but it would need to be addressed before deploying PiHub in a shared or semi-public environment. Token-based authentication on the FastAPI backend and HTTPS on the Nginx frontend would be the appropriate mitigations.

The Matter layer itself provides strong device-level security through device attestation and CASE session encryption, as described in Section 4.2.5. This means the nodes themselves cannot be trivially impersonated or spoofed on the network even if the local network is compromised.

10.3 Environmental Impact

The environmental consideration most relevant to PiHub is the power consumption of always-on hardware. The Raspberry Pi 4 draws approximately 3-7 watts under typical load, which over a full year of continuous operation amounts to roughly 26-61 kWh. While this is modest compared to most household appliances, it is a non-trivial consideration for a device that is intended to run permanently. The ESP32-C3 nodes are significantly more efficient -

the ESP32-C3 draws approximately 80-160 mA at 3.3V during active Wi-Fi operation, translating to under 0.5W per node, making their contribution to overall energy consumption negligible [7].

The choice of the ESP32-C3 as the node platform is itself an environmentally conscious decision relative to alternatives. A full single-board computer running Linux at each sensor location would consume orders of magnitude more power for the same sensing task. The ESP32-C3's RISC-V core and integrated radio subsystem are designed for IoT workloads specifically, and the on-device TinyML inference means classification results are produced locally without any additional compute load on the hub.

The local-only architecture of PiHub also eliminates the energy cost associated with cloud-hosted smart home platforms, where sensor data is continuously transmitted to and processed by remote data centres. While the per-user contribution to a data centre's energy consumption is small, it is a real cost that PiHub avoids entirely.

10.4 Accessibility

The PiHub dashboard was designed primarily for touch interaction on a fixed kiosk display, and several accessibility considerations informed the UI design decisions made during development.

Touch target sizes were kept large throughout the interface. Buttons, tiles, toggle switches, and navigation links all meet or exceed the 44x44 pixel minimum touch target size recommended by WCAG 2.1 [26], ensuring they are usable by people with limited fine motor control or larger fingers. The brightness slider uses a full-width track to maximise the touch target area for drag gestures.

Font sizes were chosen to be legible at arm's length from a touchscreen. Key sensor values - temperature, humidity, and the context badge label - are displayed at larger sizes than supporting text, creating a clear visual hierarchy that allows the most important information to be read quickly without requiring the user to lean towards the screen.

Colour is not used as the sole means of conveying information anywhere in the interface. The context badge, for example, uses both a colour modifier and a text label to indicate the current context - a user who cannot distinguish the badge colours can still read HEATING_ON, NORMAL, or WINDOW_OPEN from the label text. The LED on/off state is similarly conveyed by both colour and a text label.

Dark and light mode support was implemented via the ThemeContext system, giving users the option to switch to whichever mode provides better contrast for their visual needs. Accent colours for the sensor and LED tiles are also user-configurable, allowing individuals to choose colour combinations that work better for them if the defaults are not suitable.

11. Conclusion

11.1 Summary of Outcomes

PiHub set out to demonstrate that a fully functional, brand-independent smart home hub could be built using open standards on consumer hardware, with no cloud dependencies at runtime. That goal was achieved in full.

Two custom ESP32-C3-DevKitM-1 nodes were developed using the Esp-matter SDK and successfully commissioned onto a Matter fabric managed by python-matter-server running on a Raspberry Pi 4. The sensor node reports live temperature, humidity, and TinyML context classifications through the Matter protocol. The LED actuator node supports full RGB colour control, brightness, and on/off toggling from the dashboard. Both nodes run continuously in 3D-printed enclosures.

The full software stack is containerised and deployed via Docker Compose across two stacks on the Raspberry Pi, with a FastAPI backend, PostgreSQL database, and React/Vite frontend served via Nginx. A GitHub Actions CI/CD pipeline automates building, testing, and publishing multi-architecture Docker images to GHCR on every push to main, with parallel pytest and Vitest test suites acting as gates before the Docker build step.

The TinyML workstream produced a three-class context classifier running entirely on the ESP32-C3 using TensorFlow Lite Micro. The model - a shallow MLP trained on synthetic Gaussian data and quantised to int8 - classifies the current environmental context as HEATING_ON, NORMAL, or WINDOW_OPEN with 100% accuracy on a held-out test split of the synthetic dataset, validated qualitatively in deployment by manually inducing each context condition, and surfaces the result through a Matter attribute workaround that bypasses a confirmed CHIP SDK bug.

The React dashboard is touch-optimised for Chromium kiosk mode on the Raspberry Pi touchscreen, with automated Vitest tests covering the colour pipeline, API modules, and the LED state hook, and a full pytest suite covering the backend endpoints and Matter WebSocket client.

11.2 Evaluation Against Objectives

The table below evaluates the delivered outcomes against the objectives set out in the original project proposal. It is worth noting that the proposal described an MQTT-based architecture. Following initial research, this was replaced with the Matter protocol as the communication standard, which represented a more technically ambitious and professionally relevant approach to the same core goal.

Objective	Outcome	Status
Build a Raspberry Pi-hosted central smart home hub	Raspberry Pi 4 running a fully containerised Docker Compose stack with FastAPI, PostgreSQL, and Nginx	Met
Communicate with ESP32 nodes over Wi-Fi	Two ESP32-C3 nodes commissioned onto a Matter fabric, communicating over Matter-over-Wi-Fi	Met
Implement a temperature and humidity sensor node	DHT22 sensor node reporting live temperature, humidity, and TinyML context classification via Matter	Met

Implement an LED actuator node	WS2812 RGB LED node with full colour, brightness, and on/off control via Matter ColorControl and LevelControl clusters	Met
Build a REST API server on the Raspberry Pi	FastAPI backend with full endpoint coverage for sensor data, device management, LED control, and commissioning	Met
Build a React dashboard running in kiosk mode on the Pi touchscreen	Touch-optimised React/Vite dashboard served via Nginx, running in Chromium full-screen mode with autostart	Met
Store sensor history in a database	PostgreSQL database persisting temperature, humidity, and context classification readings with timezone-aware timestamps	Met
Containerise services using Docker	Two-stack Docker Compose architecture with automated CI/CD pipeline building and publishing images to GHCR	Exceeded
Voice assistant integration	Not implemented within the project timeline - identified in the proposal as an optional extra	Not Met
On-device intelligence (TinyML)	Three-class context classifier running on the ESP32-C3 via TensorFlow Lite Micro, not in original proposal	Exceeded
Automated testing	pytest backend suite and Vitest frontend suite integrated as CI gates, not in original proposal	Exceeded

11.3 Future Work

Several areas of the project have clear potential for further development beyond what was delivered within the project timeline.

The most natural extension would be adding support for additional Matter device types. The hub architecture is already generic - commissioning a new node and registering it in the device registry would make it immediately visible on the dashboard. Door locks, motorised blinds, and occupancy sensors are all well-supported Matter device types that would extend PiHub into a more complete smart home system without requiring significant changes to the hub software.

Thread/Border Router support would open up a wider range of low-power Matter devices. The ESP32-C3 only supports Matter over Wi-Fi, but many commercially available Matter devices use Thread as their network layer. Adding an Open Thread Border Router to the network would allow these devices to be commissioned onto the same fabric as the existing Wi-Fi nodes.

Multi-user access control was explicitly descoped in favour of the TinyML workstream, but it would be a meaningful addition for any deployment beyond a single-user home. Token-based authentication on the FastAPI backend and user account management in the dashboard would be the starting points.

The TinyML context classifier could be expanded with more classes - distinguishing between different rooms, detecting cooking activity via humidity spikes, or identifying appliance usage patterns. A larger and more varied real-world training dataset would also improve the robustness of the existing three classes beyond what synthetic Gaussian data alone can provide.

Voice assistant integration was identified in the original project proposal as a potential extra feature. Matter now has native voice assistant integration support through platforms like Apple Home and Google Home, meaning a commissioned PiHub node could in principle be controlled by a voice assistant without any additional development on the hub side.

11.4 Reflection

I came into this project expecting the backend and frontend to be the hardest parts, since those were the areas I had the most existing knowledge to build on. The firmware ended up being the hardest by a distance. Debugging embedded code is a completely different experience - there is no debugger you can easily attach, serial output is your main window into what is happening, and a bug can show up far from where it actually originated. The brightness and colour bug on the LED node is the one that sticks out most. The symptom was a white flash every time the brightness changed, but the root cause was a hardware-level side effect buried inside the LED driver. Fixing it meant understanding the full chain from Matter attribute update through to hardware output, and then making coordinated changes on both the firmware and the frontend. That kind of cross-layer debugging was something I had not done before.

The vendor cluster SDK bug was a different kind of frustrating. Hours of monitoring WebSocket messages and checking serial logs eventually led to the conclusion that the problem was not in my code at all - it was a confirmed bug in the CHIP SDK itself. That was actually a valuable experience in its own way. It taught me to be more systematic about elimination and not to assume the fault is always on your side.

The switch from anomaly detection to supervised classification for the TinyML workstream was a decision I am glad I made. The autoencoder approach felt more impressive on paper but it did not actually solve the problem I had. A model that can only tell you something is different is not that useful when you already know what the different things are. The three-class classifier is simpler, more interpretable, and does exactly what I needed.

If I was starting over, I would integrate the python-matter-server WebSocket earlier. I spent about a month trying to integrate my own backend Matter controller but ran into a lot of issues with credentials. Once I switched to using the python-matter-server, it took about two weeks to get a fully integrated Matter controller. This would have saved me a lot of time looking back and it could have allowed me to integrate Token-based authentication.

Appendix

Appendix A - Full Firmware Source Listings

https://github.com/Tremainm/ESP32C3-LED

https://github.com/Tremainm/ESP32C3-DHT22-Sensor

Appendix B - PiHub Source Code

https://github.com/Tremainm/PiHub-FYP2025

Appendix C - Docker Compose Files

https://github.com/Tremainm/PiHub-deploy

Appendix D - GitHub Actions Workflow

https://github.com/Tremainm/PiHub-FYP2025/blob/main/.github/workflows/ci.yml

Appendix E - TinyML Model Training Script

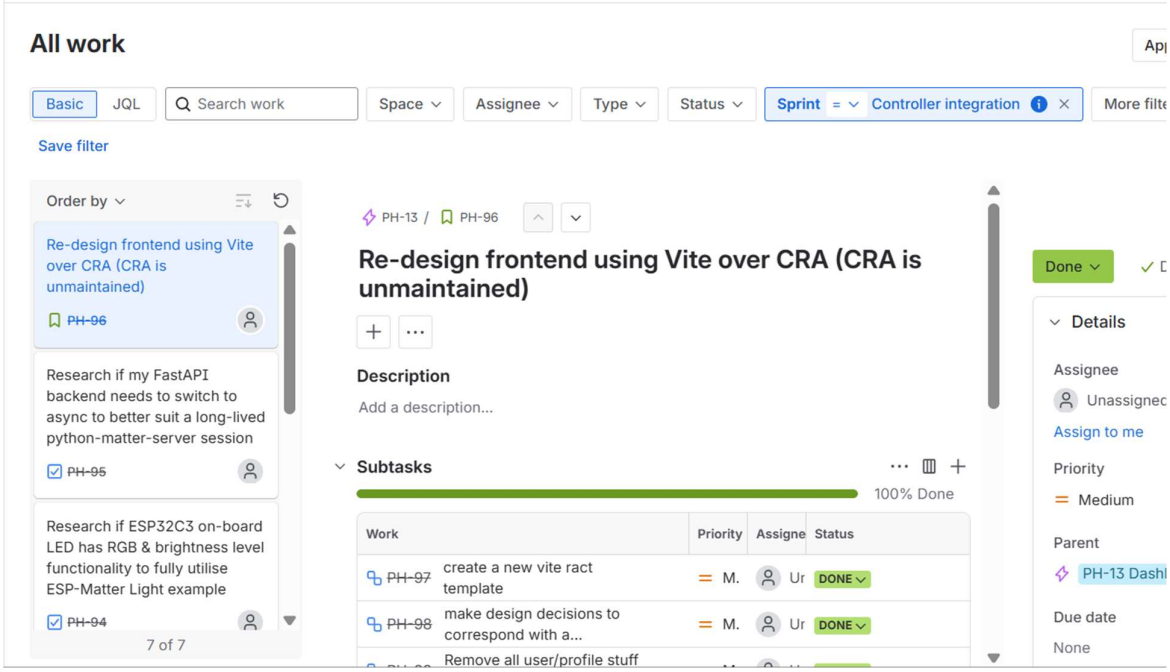
https://github.com/Tremainm/train_tinyml/blob/main/train_tinyml_context.py

Appendix F - 3D Enclosure Details

https://www.printables.com/model/1078765-raspberry-pi-touch-display-2-7-case-for-raspberry

https://www.printables.com/model/162815-esp32-c3-case

Appendix G - Jira Project Screenshot Example



Appendix H - Weekly Project Logs Example

Custom Cluster

Tuesday 31 March 2026 21:05

Overview

This week was focused on getting the context classification label from the ESP32-C3 all the way through to the frontend dashboard. The vendor cluster approach ran into a confirmed CHIP SDK bug, which took a full investigation to diagnose properly before moving to a workaround.

Vendor Cluster Investigation

The problem

After adding the custom Matter cluster (0xFC00) to the firmware, python-matter-server was receiving the attribute data from the device but silently discarding it before it ever reached the Python layer:

```
CHIP_ERROR [chip.native.DMG] Skipping AttributeDataIB
with out-of-range IDs: (1, 0x0000_FC00, 0x0000_0000)
CHIP_ERROR [chip.native.DMG] Skipping AttributeDataIB
with out-of-range IDs: (1, 0x0000_FC00, 0x0000_FFFB)
CHIP_ERROR [chip.native.DMG] Skipping AttributeDataIB
with out-of-range IDs: (1, 0x0000_FC00, 0x0000_FFF9)
CHIP_ERROR [chip.native.DMG] Skipping AttributeDataIB
with out-of-range IDs: (1, 0x0000_FC00, 0x0000_FFF8)
```

The Skipping AttributeDataIB with out-of-range IDs message comes from the CHIP C++ layer inside the container, not from python-matter-server itself. The first assumption was that the cluster was missing mandatory global attributes (ClusterRevision at 0xFFFD, FeatureMap at 0xFFFC). The low-level cluster::create() in esp-matter doesn't add these automatically, every standard cluster wrapper adds them explicitly after calling the low-level function. The vendor cluster setup was only calling the low-level one directly:

- `cluster_t *cluster = cluster::create(ep, kContextClusterId, CLUSTER_FLAG_SERVER);`

Adding the global attributes fixed the missing 0xFFFD and 0xFFFC entries:

```
cluster_t *cluster = cluster::create(ep, kContextClusterId, CLUSTER_FLAG_SERVER);
global::attribute::create_cluster_revision(cluster, 1);
global::attribute::create_feature_map(cluster, 0);
```

After reflashing and recommissioning, all six expected attributes were now appearing in the skip list:

```
Skipping AttributeDataIB with out-of-range IDs: (1,
0x0000_FC00, 0x0000_FFFD)
Skipping AttributeDataIB with out-of-range IDs: (1,
0x0000_FC00, 0x0000_FFFC)
Skipping AttributeDataIB with out-of-range IDs: (1,
0x0000_FC00, 0x0000_0000)
Skipping AttributeDataIB with out-of-range IDs: (1,
0x0000_FC00, 0x0000_FFFB)
Skipping AttributeDataIB with out-of-range IDs: (1,
```

References

[1] IoT Now, "Interoperability issues: The hidden challenges of IoT integration," [Online]. Available: https://iot-now.com/2024/06/04/144693-interoperability-issues-the-hidden-challenges-of-iot-integration/.

[2] Fierce Network, "Thread Group, ZigBee Alliance announce collaboration to reduce IoT fragmentation," [Online]. Available: https://www.fierce-network.com/tech/thread-group-zigbee-alliance-announce-collaboration-to-reduce-iot-fragmentation/.

[3] Connectivity Standards Alliance, "About the CSA," [Online]. Available: https://csa-iot.org/about/.

[4] Connectivity Standards Alliance, "Matter," [Online]. Available: https://csa-iot.org/all-solutions/matter/.

[5] Connectivity Standards Alliance, "Discovery," [Online]. Available: Available: https://handbook.buildwithmatter.com/how-it-works/discovery/.

[6] Espressif Systems, "ESP-IDF Programming Guide," release/v5.x, Espressif Systems, 2024. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/.

[7] Espressif Systems, "ESP32-C3 Series Datasheet," version 1.7, Espressif Systems, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-c3_datasheet_en.pdf.

[8] Aosong Electronics, "Digital-output relative humidity & temperature sensor/module," Datasheet, Aosong Electronics, [Online]. Available: https://www.apogeeweb.net/upload/pdf/20220126/DHT22-Aosong-Datasheet.pdf.

[9] WorldSemi, "WS2812B Intelligent Control LED Integrated Light Source," [Online]. Available: https://cdn-shop.adafruit.com/datasheets/WS2812B.pdf.

[10] FastAPI, "FastAPI," [Online]. Available: https://fastapi.tiangolo.com/.

[11] The PostgreSQL Global Development Group, "PostgreSQL Documentation," 2024. [Online]. Available: https://www.postgresql.org/docs/17/.

[12] Docker Inc., "Docker Compose documentation," 2024. [Online]. Available: https://docs.docker.com/compose/.

[13] GitHub, "GitHub Actions documentation," 2024. [Online]. Available: https://docs.github.com/en/actions.

[14] TensorFlow, "TensorFlow Lite Micro," GitHub repository, 2024. [Online]. Available: https://github.com/tensorflow/tflite-micro.

[15] DFRobot, "DHT22 wiring schematic," [Online]. Available: https://dfimg.dfrobot.com/wiki/17500/SEN0137_dht22-temperature-and-humidity-sensor_schematics_v1.0.pdf.

[16] Espressif, "ESP32-C3-DevKitM-1 Pin Layout," [Online]. Available: https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32c3/esp32-c3-devkitm-1/user_guide.html#hardware-reference.

[17] Espressif, "ESP32-C3-DevKitM-1 Schematics," [Online]. Available: https://dl.espressif.com/dl/schematics/SCH_ESP32-C3-DEVKITM-1_V1_20200915A.pdf.

[18] Home Assistant, "python-matter-server," GitHub repository, [Online]. Available: https://github.com/home-assistant-libs/python-matter-server.

[19] Python, "Asyncio," [Online]. Available: https://docs.python.org/3/library/asyncio.html.

[20] Vitest, "Vitest," [Online]. Available: https://vitest.dev/.

[21] Raspberry Pi Forums, "Scroll Touch Selects Text Instead," [Online]. Available: https://forums.raspberrypi.com/viewtopic.php?p=2319656&hilit=mouse+emulation#p2314608.

[22] drandreww, "ESP32-C3 Case," Printables, [Online]. Available: https://www.printables.com/model/162815-esp32-c3-case.

[23] Iain White, "Raspberry Pi Touch Display 2, 7" Case for Raspberry Pi," Printables, [Online]. Available: https://www.printables.com/model/1078765-raspberry-pi-touch-display-2-7-case-for-raspberry.

[24] RonnyS, "7" Touchscreen Case for Raspberry Pi 4," Printables, [Online]. Available: https://www.printables.com/model/44218-7-touchscreen-case-for-raspberry-pi-4-internal-fan.

[25] European Parliament, "Regulation (EU) 2016/679 (General Data Protection Regulation)," [Online]. Available: https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32016R0679.

[26] W3C, "Understanding Success Criterion 2.5.5: Target Size," WCAG 2.1, [Online]. Available: https://www.w3.org/TR/WCAG21/#target-size.